



UNIVERSIDADE
FEDERAL
DE PERNAMBUCO

Time | UFPE

Thawan

Contents

1 combinatorial	2	5 math	11
1.1 binomial-inversion	2	5.1 fast-fourier-transform	11
1.2 catalan	2	5.2 fast-walsh-hadamard-transform	11
1.3 lucas	2	5.3 modulo-transform-technique	11
1.4 permutation-combination	2	5.4 number-theoretic-transform	12
1.5 stars-and-bars	2	5.5 gaussian-elimination	12
2 dynamic-programming	2	5.6 josephus	13
2.1 bounded-subset-sum	2	5.7 berlekamp-massey	13
2.2 divide-and-conquer	3	5.8 kth-term	13
2.3 knuth	3	5.9 matrix	13
2.4 sum-over-subsets	3	5.10 min-plus-matrix	13
2.5 sum-over-supersets	3	5.11 nim	13
3 geometry	3	5.12 xor-basis	14
3.1 point	3	6 number-theory	14
3.2 polygons	3	6.1 harmonic-lemma	14
3.3 primitives	4	6.2 mobius	14
4 graph	4	6.3 chinese-remainder-theorem	14
4.1 articulations	4	6.4 extended-gcd	14
4.2 block-cut-tree	4	6.5 mod-log	14
4.3 bridge-tree	4	6.6 mod-pow	15
4.4 bridges	5	6.7 multi-extended-gcd	15
4.5 cycle	5	6.8 linear-sieve	15
4.6 dominator-tree	5	6.9 miller-rabin	15
4.7 tarjan	5	6.10 segmented-sieve	15
4.8 bellman-ford	6	6.11 totient	15
4.9 dijkstra	6	7 search	15
4.10 floyd-warshall	6	7.1 ternary-search	15
4.11 johnson	6	8 sheet	16
4.12 dinic	6	9 square-root	16
4.13 min-cost-k-flow	7	9.1 decomposition	16
4.14 hierholzer	7	9.2 mo	16
4.15 hopcroft-karp	7	9.3 mo-3d	16
4.16 hungarian	8	10 string	16
4.17 kuhn	8	10.1 aho-corasick	16
4.18 maximum-clique	8	10.2 deterministic-finite-automaton	17
4.19 binary-lifting	9	10.3 dynamic-hashier	17
4.20 centroid-decomposition	9	10.4 hasher	17
4.21 heavy-light-decomposition	9	10.5 longest-prefix-suffix	17
4.22 lazy-heavy-light-decomposition	9	10.6 manacher	17
4.23 leaf-peeling	10	10.7 palindromic-tree	18
4.24 reversible-heavy-light-decomposition	10	10.8 suffix-array	18

10.9	suffix-automaton	19
10.10	tree-suffix-array	19
10.11	trie	20
10.12	z-array	20
11	structures	20
11.1	binary-indexed-tree	20
11.2	binary-indexed-tree-2d	20
11.3	range-binary-indexed-tree	20
11.4	sparse-binary-indexed-tree	21
11.5	cover-tree	21
11.6	k-cover-tree	21
11.7	xor-cover-tree	21
11.8	coprime-set	22
11.9	interval-set	22
11.10	range-set	22
11.11	venice-set	22
11.12	disjoint-set-union	23
11.13	disjoint-set-union-2d	23
11.14	dynamic-connectivity	23
11.15	rollback-disjoint-set-union	23
11.16	li-chao-tree	23
11.17	line-container	24
11.18	dynamic-merge-sort-tree	24
11.19	merge-sort-tree	24
11.20	hash-table	24
11.21	ordered-set	24
11.22	lazy-segment-tree	24
11.23	lazy-sparse-segment-tree	25
11.24	persistent-segment-tree	25
11.25	recursive-segment-tree	26
11.26	reversible-segment-tree	26
11.27	segment-tree	26
11.28	segment-tree-2d	26
11.29	sparse-persistent-segment-tree	26
11.30	sparse-segment-tree	27
11.31	sparse-table	27
11.32	sparse-table-2d	27
11.33	implicit-treap	27
11.34	treap	28
11.35	wavelet-matrix	28
12	extra	29
12.1	random.cpp	29
12.2	permutation-int.cpp	29
12.3	pragmas.cpp	29
12.4	submasks.cpp	29
12.5	offset.cpp	29
12.6	macros.cpp	29

1 combinatorial

1.1 binomial-inversion

```
// Binomial Inversion: transforms subset group counts into exact
// frequency counts
// Time:  $O(n^2)$  | Space:  $O(n)$ 
// Formula:  $exact[i] = \sum_{j=i}^n (-1)^{j-i} * C(j, i) * subsets[j]$ 
vector<int> binomial_inversion(const vector<int> &subsets) {
    int n = subsets.size() - 1;
    vector<int> exact(n + 1);
    for (int i = 0; i <= n; ++i) {
```

```
        int count = 0;
        for (int j = i; j <= n; ++j) {
            int term = (C(j, i) * subsets[j]) % mod;
            if ((j - i) & 1) {
                count = (count - term + mod) % mod;
            } else {
                count = (count + term) % mod;
            }
        }
        exact[i] = count;
    }
    return exact;
}
```

1.2 catalan

```
// Catalan (Ballot Theorem): number of grid paths from (0,0) to (1,r)
// staying x >= y at all times
// Time:  $O(1)$  | Space:  $O(1)$ 
// Note: requires C(n, k) | also counts valid parenthesis prefixes (1
// open, r close)
int catalan(int l, int r) {
    if ((l < 0 or r < 0) or l < r) return 0;
    return (C(l + r, l) - C(l + r, l + 1) + mod) % mod;
}
```

1.3 lucas

```
int lucas(int n, int k) {
    if (k == 0 or k == n) return 1;
    if (k > n) return 0;
    return (lucas(n / mod, k / mod) * C(n % mod, k % mod)) % mod;
}
```

1.4 permutation-combination

```
int P(int n, int k) {
    if (k < 0 or n < k) return 0;
    return fact[n] * invf[n - k] % mod;
}
```

```
int C(int n, int k) {
    if (k < 0 or n < k) return 0;
    return P(n, k) * invf[k] % mod;
}
```

1.5 stars-and-bars

```
// Stars and Bars: number of ways to put n identical items into k
// distinct bins
// Time:  $O(1)$  | Space:  $O(1)$ 
// Note: requires C(n, k) function | counts non-negative integer
// solutions to  $x_1 + \dots + x_k = n$ 
int stars_andBars(int n, int k) {
    if (n < 0 or k < 0) return 0;
    if (k == 0) return n == 0;
    return C(n + k - 1, k - 1);
}
```

2 dynamic-programming

2.1 bounded-subset-sum

```
// Bounded Subset Sum: maximizes subset sum without exceeding the
// target
// Time:  $O(N * \max(W))$  | Space:  $O(\max(W))$ 
// Note: efficient for huge targets with small weights
int bounded_subset_sum(vector<int> &w, int target) {
    int k = 0, s = 0;
    while (k < w.size() and s + w[k] <= target) s += w[k++];

    if (k == w.size() or s == target) return s;

    int bound = *max_element(w.begin(), w.end());
    int offset = target - bound;

    vector<int> prev(2 * bound + 1, -1);
    prev[s - offset] = k;

    for (int i = k; i < w.size(); ++i) {
        vector<int> curr = prev;

        // insert
        for (int x = 0; x < bound; ++x) if (prev[x] != -1) {
            curr[x + w[i]] = max(curr[x + w[i]], prev[x]);
        }

        // remove
        for (int x = 2 * bound; x > bound; --x) if (curr[x] >
            prev[x]) {
            for (int j = max(0ll, prev[x]); j < curr[x]; ++j) {
                curr[x - w[j]] = max(curr[x - w[j]], j);
            }
        }

        swap(prev, curr);
    }
}
```

```

for (int x = bound; x >= 0; --x) if (prev[x] != -1) {
    return x + offset;
}
return 0;
}

```

2.2 divide-and-conquer

```

int divide_and_conquer() {
    vector<int> prev(n + 1, inf);
    vector<int> curr(n + 1, inf);
    auto compute = [&](auto &self, int l, int r, int opt_l, int
        opt_r) -> void {
        if (l > r) return;
        int m = (l + r) / 2;
        curr[m] = inf;
        int opt = opt_r;
        for (int i = max(opt_l, m); i <= min(opt_r, n - 1); ++i) if
            (prev[i + 1] != inf) {
                int val = cost(m, i) + prev[i + 1];
                if (val < curr[m]) {
                    curr[m] = val;
                    opt = i;
                }
            }
        self(self, l, m - 1, opt_l, opt);
        self(self, m + 1, r, opt, opt_r);
    };
    for (int i = 1; i <= n; ++i) {
        curr[i] = cost(i, n);
    }
    for (int i = 2; i <= k; ++i) {
        swap(prev, curr);
        compute(compute, 1, n, 1, n);
    }
    return curr[1];
}

```

2.3 knuth

```

int knuth() {
    vector<vector<int>> val(n + 1, vector<int>(n + 1));
    vector<vector<int>> opt(n + 1, vector<int>(n + 1));
    for (int i = 1; i <= n; ++i) {
        val[i][i] = cost(i, i);
        opt[i][i] = i;
    }
    for (int len = 2; len <= n; ++len) {
        for (int l = 1, r = len; r <= n; ++l, ++r) {
            val[l][r] = inf;
            opt[l][r] = opt[l][r - 1];
            for (int k = opt[l][r - 1]; k <= min(r - 1, opt[l +
                1][r]); ++k) {
                int curr = val[l][k] + val[k + 1][r] + cost(l, r);
                if (curr < val[l][r]) {
                    val[l][r] = curr;
                    opt[l][r] = k;
                }
            }
        }
    }
    return val[1][n];
}

```

2.4 sum-over-subsets

```

vector<int> sum_over_subsets(int n, vector<int> f) {
    int full = (1 << n) - 1;
    for (int i = 0; i < n; ++i) {
        int bit = 1 << i;
        for (int mask = 0; mask <= full; ++mask) if (mask & bit) {
            f[mask] += f[mask ^ bit];
        }
    }
    return f;
}

```

2.5 sum-over-supersets

```

vector<int> sum_over_supersets(int n, vector<int> f) {
    int full = (1 << n) - 1;
    for (int i = 0; i < n; ++i) {
        int bit = 1 << i;
        for (int mask = 0; mask <= full; ++mask) if (not (mask &
            bit)) {
            f[mask] += f[mask | bit];
        }
    }
    return f;
}

```

3 geometry

3.1 point

```

#define x real()
#define y imag()

```

```

template <typename T>
using point = complex<T>;

```

```

template <typename T>
T dot(point<T> p, point<T> q) {
    return (conj(p) * q).x;
}

```

```

template <typename T>
T cross(point<T> p, point<T> q) {
    return (conj(p) * q).y;
}

```

```

// Directed Angle: shortest angle from vector v to vector w
// Note: returns radians in [-pi, pi] | negative result means w is to
// the right

```

```

template <typename T>
float angle(point<T> v, point<T> w) {
    return atan2(cross(v, w), dot(v, w));
}

```

```

// Half: determines which half-plane a point lies in relative to ray v
// Note: returns 0 for [0, pi] relative to v (including origin), 1
// for [pi, 2pi)

```

```

template <typename T>
bool half(point<T> p, point<T> v) {
    T c = cross(v, p);
    T d = dot(v, p);
    return c < 0 or (c == 0 and d < 0);
}

```

```

// Polar Sort: sorts points counter-clockwise around an origin
// starting at reference ray v
// Time: O(N log N) | Space: O(1)
// Note: colinear points sorted by distance | exact integer arithmetic

```

```

template <typename T>
void polar_sort(vector<point<T>> &p, point<T> o = {0, 0}, point<T> v
    = {1, 0}) {
    sort(p.begin(), p.end(), [&](point<T> a, point<T> b) {
        a = a - o, b = b - o;
        bool ha = half(a, v), hb = half(b, v);
        if (ha != hb) return ha < hb;
        T c = cross(a, b);
        if (c != 0) return c > 0;
        return norm(a) < norm(b);
    });
}

```

3.2 polygons

```

float area(vector<point> &p) {
    int n = p.size();
    float a = 0;
    for (int i = 0; i < n; ++i) {
        int j = (i + 1) % n;
        a += p[i].x * p[j].y - p[j].x * p[i].y;
    }
    return a / 2;
}

```

```

vector<point> convex_hull(vector<point> &p, bool collinear) {
    sort(p.begin(), p.end());
    p.erase(unique(p.begin(), p.end(), p.end()));
}

```

```

int n = p.size();
if (n <= 2) return p;

```

```

bool all_collinear = true;
line h = {p[0], p[1]};
for (int i = 2; i < n and all_collinear; ++i) all_collinear &=
    h.orientation(p[i]) == 0;
if (all_collinear) return collinear ? p : vector<point>{p[0], p[n
    - 1]};

```

```

vector<point> lower;
for (int i = 0; i < n; ++i) {
    while (lower.size() >= 2) {
        line l = {lower[lower.size() - 2], lower[lower.size() -
            1]};
        int o = l.orientation(p[i]);
        if (o == -1 or (o == 0 and collinear)) break;
        lower.pop_back();
    }
    lower.push_back(p[i]);
}

```

```

vector<point> upper;
for (int i = n - 1; i >= 0; --i) {
    while (upper.size() >= 2) {
        line l = {upper[upper.size() - 2], upper[upper.size() -
            1]};
        int o = l.orientation(p[i]);
        if (o == -1 or (o == 0 and collinear)) break;
        upper.pop_back();
    }
    upper.push_back(p[i]);
}

```

```

lower.pop_back();
upper.pop_back();

```

```

vector<point> hull = lower;
hull.insert(hull.end(), upper.begin(), upper.end());
return hull;
}

```

3.3 primitives

```

struct point {
    float x, y;

    bool operator==(const point &other) const {
        return x == other.x and y == other.y;
    }

    bool operator<(const point &other) const {
        return tie(x, y) < tie(other.x, other.y);
    }

    point operator-(const point &other) const {
        return {x - other.x, y - other.y};
    }

    static float dist(point &a, point &b) {
        float dx = a.x - b.x;
        float dy = a.y - b.y;
        return sqrt(dx * dx + dy * dy);
    }

    static float dot(point &a, point &b) {
        return a.x * b.x + a.y * b.y;
    }
};

struct line {
    point a, b;

    int orientation(point &p) {
        point v = b - a;
        point w = p - a;

        float c = v.x * w.y - v.y * w.x;
        if (c > 0) return -1;
        if (c < 0) return +1;
        return 0;
    }

    bool between(point &p) {
        if (orientation(p) != 0) return false;

        bool bx = min(a.x, b.x) <= p.x and p.x <= max(a.x, b.x);
        bool by = min(a.y, b.y) <= p.y and p.y <= max(a.y, b.y);
        return bx and by;
    }

    point project(point &p) {
        point v = b - a;
        point w = p - a;

        float t = point::dot(v, w) / point::dot(v, v);
        if (t < 0) t = 0;
        if (t > 1) t = 1;

        return {a.x + t * v.x, a.y + t * v.y};
    }

    float dist(point &p) {
        point q = project(p);
        return point::dist(p, q);
    }

    line normal() {
        point v = b - a;
        v = {-v.y, v.x};

        point c = {(a.x + b.x) / 2, (a.y + b.y) / 2};
        point d = {c.x + v.x, c.y + v.y};
        return {c, d};
    }

    static float angle(line &s, line &t) {
        point v = s.b - s.a;
        point w = t.b - t.a;

        float c = point::dot(v, w) / sqrt(point::dot(v, v) *
            point::dot(w, w));
        if (c < -1) c = -1;
        if (c > 1) c = 1;

        return acos(c);
    }

    static bool intersect(line &s, line &t) {
        if (s.between(t.a) or s.between(t.b)) return true;
        if (t.between(s.a) or t.between(s.b)) return true;

        if (s.orientation(t.a) == s.orientation(t.b)) return false;
        if (t.orientation(s.a) == t.orientation(s.b)) return false;

        return true;
    }
}

```

```

static float dist(line &s, line &t) {
    if (line::intersect(s, t)) return 0;
    return min({s.dist(t.a), s.dist(t.b), t.dist(s.a),
        t.dist(s.b)});
}
};

```

4 graph

4.1 articulations

```

vector<int> articulations(int one, vector<vector<int>> &g) {
    int n = g.size(), timer = 1;
    vector<int> in(n), low(n);
    vector<bool> art(n);

    auto dfs = [&](auto &self, int a, int p) -> void {
        in[a] = low[a] = timer++;
        int c = 0;
        for (int b : g[a]) if (b != p) {
            if (in[b] != 0) {
                low[a] = min(low[a], in[b]);
            } else {
                c++;
                self(self, b, a);
                low[a] = min(low[a], low[b]);
                if (a != p and low[b] >= in[a]) art[a] = true;
            }
        }
        if (a == p and c > 1) art[a] = true;
    };

    vector<int> points;
    for (int i = one; i < n; ++i) {
        if (in[i] == 0) dfs(dfs, i, i);
        if (art[i]) points.push_back(i);
    }
    return points;
}

```

4.2 block-cut-tree

```

// Block-Cut Tree: decomposes an undirected graph into biconnected
// components
// Time: O(V + E) | Space: O(V + E)
// Note: returns a tree where nodes [0, N) are original vertices and
// nodes >= N are blocks
vector<vector<int>> block_cut_tree(vector<vector<int>> &g) {
    int n = g.size(), t = 0;
    vector<int> tin(n, -1), low(n), st;
    vector<vector<int>> bct(n); bct.reserve(2 * n);
    auto dfs = [&](auto &self, int a, int p) -> void {
        tin[a] = low[a] = t++;
        st.push_back(a);
        bool parent = true;
        for (int b : g[a]) {
            if (p == b and parent) {
                parent = false;
                continue;
            }
            if (tin[b] != -1) {
                low[a] = min(low[a], tin[b]);
            } else {
                self(self, b, a);
                low[a] = min(low[a], low[b]);
                if (tin[a] <= low[b]) {
                    int i = bct.size();
                    bct.emplace_back();
                    bct[i].push_back(a);
                    bct[a].push_back(i);
                    while (true) {
                        int c = st.back(); st.pop_back();
                        bct[i].push_back(c);
                        bct[c].push_back(i);
                        if (b == c) break;
                    }
                }
            }
        }
    };

    for (int i = 0; i < n; ++i) {
        if (tin[i] == -1) {
            dfs(dfs, i, -1);
        }
    }
    return bct;
}

```

4.3 bridge-tree

```

// Bridge Tree: condenses 2-edge-connected components into single
// nodes
// Time: O(V + E) | Space: O(V + E)
// Note: returns a tree and an id array where id[a] is the new node
// id of original vertex a
pair<vector<vector<int>>, vector<int>>
bridge_tree(vector<vector<int>> &g) {
    int n = g.size(), t = 0;
}

```

```

vector<int> tin(n, -1), low(n), id(n), st;
vector<vector<int>> bt;
auto dfs = [&](auto &self, int a, int p) -> void {
    tin[a] = low[a] = t++;
    st.push_back(a);
    bool parent = true;
    for (int b : g[a]) {
        if (b == p and parent) {
            parent = false;
            continue;
        }
        if (tin[b] != -1) {
            low[a] = min(low[a], tin[b]);
        } else {
            self(self, b, a);
            low[a] = min(low[a], low[b]);
        }
    }
    if (low[a] == tin[a]) {
        int i = bt.size();
        bt.emplace_back();
        while (true) {
            int c = st.back(); st.pop_back();
            id[c] = i;
            if (c == a) break;
        }
    }
};
for (int i = 0; i < n; ++i) {
    if (tin[i] == -1) {
        dfs(dfs, i, -1);
    }
}
for (int a = 0; a < n; ++a) {
    for (int b : g[a]) {
        if (id[a] != id[b]) {
            bt[id[a]].push_back(id[b]);
        }
    }
}
return {bt, id};
}

```

4.4 bridges

```

vector<pair<int, int>> bridges(int one, vector<vector<int>> &g) {
    int n = g.size(), timer = 1;
    vector<int> in(n), low(n);
    vector<pair<int, int>> e;

    auto dfs = [&](auto &self, int a, int p) -> void {
        in[a] = low[a] = timer++;
        for (int b : g[a]) if (b != p) {
            if (in[b] != 0) {
                low[a] = min(low[a], in[b]);
            } else {
                self(self, b, a);
                low[a] = min(low[a], low[b]);
                if (in[a] < low[b]) e.emplace_back(a, b);
            }
        }
    };

    for (int i = one; i < n; ++i) if (in[i] == 0) dfs(dfs, i, i);
    return e;
}

```

4.5 cycle

```

vector<int> find_cycle(bool one, bool is_directed,
    vector<vector<int>> &g) {
    int n = g.size();
    vector<int> state(n), parent(n);
    vector<int> cycle;
    auto dfs = [&](auto &self, int a, int p) -> bool {
        state[a] = 1;
        for (int b : g[a]) if (b != p or is_directed) {
            if (state[b] == 0) {
                parent[b] = a;
                if (self(self, b, a)) return true;
            } else if (state[b] == 1) {
                int c = a;
                while (c != b) {
                    cycle.push_back(c);
                    c = parent[c];
                }
                cycle.push_back(b);
                reverse(cycle.begin(), cycle.end());
                return true;
            }
        }
        state[a] = 2;
        return false;
    };
    for (int i = one; i < n; ++i) if (state[i] == 0) {
        if (dfs(dfs, i, -1)) return cycle;
    }
    return {};
}

```

4.6 dominator-tree

```

// Dominator Tree: computes immediate dominators in directed graphs
// Time: build  $O(V \log V + E)$  | Space:  $O(V + E)$ 
// Note: returns the tree where node a immediately dominates its children
struct dominator_dsu {
    vector<int> p, c;
    vector<int> &x;

    dominator_dsu(int n, vector<int> &x): p(n), c(n), x(x) {
        iota(p.begin(), p.end(), 0);
        iota(c.begin(), c.end(), 0);
    }

    int find(int a) {
        if (p[a] == a) return c[a];
        int r = p[a];
        if (p[r] != r) {
            find(r);
            if (x[c[r]] < x[c[a]]) c[a] = c[r];
            p[a] = p[r];
        }
        return c[a];
    }

    void unite(int a, int b) {
        p[b] = a;
    }
};

vector<vector<int>> dominator_tree(int s, vector<vector<int>> &g) {
    int n = g.size();
    vector<int> tin(n, -1), inv(n), parent(n);
    int t = 0;
    auto dfs = [&](auto &self, int a) -> void {
        tin[a] = t;
        inv[t] = a;
        t++;
        for (int b : g[a]) if (tin[b] == -1) {
            parent[b] = a;
            self(self, b);
        }
    };
    dfs(dfs, s);
    vector<vector<int>> rev(n);
    for (int a = 0; a < n; ++a) {
        for (int b : g[a]) rev[b].push_back(a);
    }
    vector<int> semi(n), dom(n);
    for (int a = 0; a < n; ++a) semi[a] = tin[a];
    vector<vector<int>> bucket(n);
    dominator_dsu dsu(n, semi);
    for (int i = t - 1; i >= 1; --i) {
        int a = inv[i];
        for (int b : rev[a]) if (tin[b] != -1) {
            semi[a] = min(semi[a], semi[dsu.find(b)]);
        }
        bucket[inv[semi[a]]].push_back(a);
        int p = parent[a];
        dsu.unite(p, a);
        for (int b : bucket[p]) {
            dom[b] = p;
            int c = dsu.find(b);
            if (semi[c] < semi[b]) dom[b] = c;
        }
        bucket[p].clear();
    }
    vector<vector<int>> dt(n);
    for (int i = 1; i < t; ++i) {
        int a = inv[i];
        if (dom[a] != inv[semi[a]]) dom[a] = dom[dom[a]];
        dt[dom[a]].push_back(a);
    }
    return dt;
}

```

4.7 tarjan

```

vector<vector<int>> tarjan(int r, vector<vector<int>> &g) {
    int n = g.size(), timer = 1;
    vector<vector<int>> components;
    vector<int> id(n), low(n);
    stack<int> trail;
    vector<bool> in(n);

    auto dfs = [&](auto &self, int a) -> void {
        id[a] = low[a] = timer++;
        trail.push(a);
        in[a] = true;
        for (int b : g[a]) {
            if (id[b] == 0) {
                self(self, b);
                low[a] = min(low[a], low[b]);
            } else if (in[b]) {
                low[a] = min(low[a], id[b]);
            }
        }
        if (id[a] == low[a]) {
            vector<int> component;
            while (true) {

```

```

        int b = trail.top(); trail.pop();
        in[b] = false;
        component.push_back(b);
        if (b == a) break;
    }
    components.push_back(component);
}
};

for (int a = r; a < n; ++a) {
    if (id[a] == 0) {
        dfs(dfs, a);
    }
}

reverse(components.begin(), components.end());
return components;
}

pair<vector<vector<int>>, vector<int>> condensed_graph(int r,
vector<vector<int>> &g) {
    int n = g.size();
    vector<vector<int>> components = tarjan(r, g);
    int m = components.size();

    vector<int> id(n);
    for (int i = 0; i < m; ++i) {
        for (int a : components[i]) {
            id[a] = i;
        }
    }

    vector<vector<int>> c(m);
    for (int a = r; a < n; ++a) {
        for (int b : g[a]) if (id[a] != id[b]) {
            c[id[a]].push_back(id[b]);
        }
    }

    for (int i = 0; i < m; ++i) {
        sort(c[i].begin(), c[i].end());
        c[i].erase(unique(c[i].begin(), c[i].end()), c[i].end());
    }
    return {c, id};
}

```

4.8 bellman-ford

```

vector<int> bellman_ford(int s, int n, vector<array<int, 3>> &e) {
    vector<int> d(n, inf);
    d[s] = 0;
    for (int i = 1; i < n; ++i) {
        bool chd = false;
        for (auto [a, b, w] : e) if (d[a] != inf and d[a] + w < d[b]) {
            d[b] = d[a] + w;
            chd = true;
        }
        if (not chd) break;
    }
    for (auto [a, b, w] : e) if (d[a] != inf and d[a] + w < d[b])
        return {};
    return d;
}

vector<int> shortest_path_faster_algorithm(int s,
vector<vector<pair<int, int>>> &g) {
    int n = g.size();
    vector<int> d(n, inf), c(n);
    vector<bool> in(n);
    queue<int> q;
    d[s] = 0;
    in[s] = true;
    q.push(s);
    while (q.size()) {
        int a = q.front(); q.pop();
        in[a] = false;
        for (auto [b, w] : g[a]) if (d[a] + w < d[b]) {
            d[b] = d[a] + w;
            if (not in[b]) {
                c[b] += 1;
                if (c[b] >= n) return {};
                in[b] = true;
                q.push(b);
            }
        }
    }
    return d;
}

```

4.9 dijkstra

```

vector<int> dijkstra(int s, vector<vector<pair<int, int>>> &g) {
    int n = g.size();
    vector<int> d(n, inf);
    priority_queue<pair<int, int>, vector<pair<int, int>>,
greater<pair<int, int>>> pq;
    d[s] = 0;
    pq.emplace(0, s);
    while (pq.size()) {
        auto [x, a] = pq.top(); pq.pop();

```

```

        if (d[a] < x) continue;
        for (auto [b, w] : g[a]) if (d[a] + w < d[b]) {
            d[b] = d[a] + w;
            pq.emplace(d[b], b);
        }
    }
    return d;
}

```

4.10 floyd-warshall

```

vector<vector<int>> floyd_warshall(int n, vector<vector<int>>> &g) {
    vector<vector<int>> d = g;
    for (int i = 0; i < n; ++i) {
        d[i][i] = 0;
    }
    for (int k = 0; k < n; ++k) {
        for (int i = 0; i < n; ++i) if (d[i][k] != inf) {
            for (int j = 0; j < n; ++j) if (d[k][j] != inf) {
                d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
            }
        }
    }
    return d;
}

```

4.11 johnson

```

vector<vector<int>> johnson(vector<vector<pair<int, int>>> &g) {
    int n = g.size();
    g.emplace_back();
    for (int i = 0; i < n; ++i) {
        g[n].emplace_back(i, 0);
    }
    vector<int> h = shortest_path_faster_algorithm(n, g);
    g.pop_back();
    if (h.empty()) return {};
    h.pop_back();
    for (int i = 0; i < n; ++i) {
        for (auto &[j, w] : g[i]) {
            w += h[i] - h[j];
        }
    }
    vector<vector<int>> d;
    for (int i = 0; i < n; ++i) {
        d.push_back(dijkstra(i, g));
        for (int j = 0; j < n; ++j) {
            if (d[i][j] != inf) d[i][j] -= h[i] - h[j];
        }
    }
    return d;
}

```

4.12 dinic

```

struct dinic {
    struct edge {
        int b, w, i;
        bool h;
    };

    int n;
    vector<vector<edge>> g;
    vector<int> l, p;

    dinic(int n): n(n), g(n), l(n), p(n) {}

    void directed(int a, int b, int w) {
        g[a].emplace_back(b, w, g[b].size(), true);
        g[b].emplace_back(a, 0, g[a].size() - 1, false);
    }

    void undirected(int a, int b, int w) {
        g[a].emplace_back(b, w, g[b].size(), true);
        g[b].emplace_back(a, w, g[a].size() - 1, true);
    }

    bool bfs(int s, int t) {
        fill(l.begin(), l.end(), -1);
        l[s] = 0;
        queue<int> q;
        q.push(s);

        while (not q.empty()) {
            int a = q.front(); q.pop();
            for (auto &[b, w, i, h] : g[a]) {
                if (w > 0 and l[b] < 0) {
                    l[b] = l[a] + 1;
                    q.push(b);
                }
            }
        }
        return l[t] >= 0;
    }

    int dfs(int a, int t, int f) {
        if (a == t or f == 0) return f;

        for (int &k = p[a]; k < g[a].size(); ++k) {
            auto &[b, w, i, h] = g[a][k];
            if (w > 0 and l[b] == l[a] + 1) {

```

```

        if (int d = dfs(b, t, min(f, w))) {
            w -= d;
            g[b][i].w += d;
            return d;
        }
    }
}
return 0;
}

int flow(int s, int t) {
    int f = 0;
    while (bfs(s, t)) {
        fill(p.begin(), p.end(), 0);
        int d = 0;
        while (d = dfs(s, t, inf)) {
            f += d;
        }
    }
    return f;
}

bool on_cut(int a) {
    return l[a] >= 0;
}

vector<pair<int, int>> saturated_edges() {
    vector<pair<int, int>> e;
    for (int a = 0; a < n; ++a) {
        for (auto [b, w, i, h] : g[a]) if (h and w == 0) {
            e.push_back({a, b});
        }
    }
    return e;
}
};

4.13 min-cost-k-flow
// Min Cost K-Flow: successive shortest path with potentials
// Time: O(F * E log V) where F is flow | Space: O(V + E)
// Note: initial costs MUST be >= 0
struct min_cost_k_flow {
    struct edge {
        int b, i, w, c;
        bool o;
    };

    int n;
    vector<vector<edge>> g;
    vector<int> d, h, p, e;

    min_cost_k_flow(int n): n(n), g(n), d(n), h(n), p(n), e(n) {}

    void directed(int a, int b, int w, int c) {
        g[a].emplace_back(b, g[b].size(), w, c, true);
        g[b].emplace_back(a, g[a].size() - 1, 0, -c, false);
    }

    bool dijkstra(int s, int t) {
        fill(d.begin(), d.end(), inf);
        priority_queue<pair<int, int>, vector<pair<int, int>>,
            greater<pair<int, int>>> pq;
        d[s] = 0;
        pq.emplace(0, s);
        while (pq.size()) {
            auto [x, a] = pq.top(); pq.pop();
            if (d[a] < x) continue;

            for (int i = 0; i < g[a].size(); ++i) {
                auto [b, j, w, c, o] = g[a][i];
                if (w <= 0) continue;

                int v = d[a] + c + h[a] - h[b];
                if (v < d[b]) {
                    d[b] = v;
                    p[b] = a;
                    e[b] = i;
                    pq.emplace(d[b], b);
                }
            }
        }
        return d[t] != inf;
    }

    pair<int, int> cost_flow(int s, int t, int k = inf) {
        int flow = 0, cost = 0;
        fill(h.begin(), h.end(), 0);
        while (flow < k and dijkstra(s, t)) {
            for (int a = 0; a < n; ++a) if (d[a] != inf) {
                h[a] += d[a];
            }

            int push = k - flow;
            for (int a = t; a != s; a = p[a]) {
                int b = p[a], i = e[a];
                push = min(push, g[b][i].w);
            }
            for (int a = t; a != s; a = p[a]) {
                int b = p[a], i = e[a], j = g[b][i].i;
                g[b][i].w -= push;

```

```

                g[a][j].w += push;
                cost += g[b][i].c * push;
            }
            flow += push;
        }
        return {flow, cost};
    }

    vector<pair<int, int>> saturated_edges() {
        vector<pair<int, int>> edges;
        for (int a = 0; a < n; ++a) {
            for (auto [b, i, w, c, o] : g[a]) if (w == 0 and o) {
                edges.push_back({a, b});
            }
        }
        return edges;
    }
};

4.14 hierholzer
// Undirected Eulerian Path: Hierholzer's algorithm for paths and
// circuits
// Time: O(V + E) | Space: O(V + E)
// Note: returns sequence of vertices or empty if impossible | s must
// be a valid start node
vector<int> undirected_eulerian_path(int s, vector<vector<pair<int,
    int>>> &g, int e) {
    int n = g.size();
    int odd = 0;
    for (int i = 0; i < n; ++i) {
        if (g[i].size() & 1) odd++;
    }
    if (odd != 0 and odd != 2) return {};
    if (odd == 2 and (g[s].size() & 1) == 0) return {};
    vector<bool> used(e + 1);
    vector<int> next(n), path;
    auto hierholzer = [&](auto &self, int a) -> void {
        while (next[a] < g[a].size()) {
            auto [b, id] = g[a][next[a]];
            next[a]++;
            if (not used[id]) {
                used[id] = true;
                self(self, b);
            }
        }
        path.push_back(a);
    };
    hierholzer(hierholzer, s);
    if (path.size() != e + 1) return {};
    reverse(path.begin(), path.end());
    return path;
}

// Directed Eulerian Path: Hierholzer's algorithm for paths and
// circuits
// Time: O(V + E) | Space: O(V + E)
// Note: returns sequence of vertices or empty if impossible | s must
// be a valid start node
vector<int> directed_eulerian_path(int s, vector<vector<int>> &g, int
    e) {
    int n = g.size();
    vector<int> in(n), out(n);
    for (int a = 0; a < n; ++a) {
        out[a] = g[a].size();
        for (int b : g[a]) {
            in[b]++;
        }
    }
    int start = 0, end = 0;
    for (int i = 0; i < n; ++i) {
        if (out[i] - in[i] == +1) start++;
        else if (out[i] - in[i] == -1) end++;
        else if (out[i] - in[i] != 0) return {};
    }
    if (not ((start == 0 and end == 0) or (start == 1 and end == 1)))
        return {};
    if (start == 1 and out[s] - in[s] != +1) return {};
    vector<int> next(n), path;
    auto hierholzer = [&](auto &self, int a) -> void {
        while (next[a] < g[a].size()) {
            int b = g[a][next[a]];
            next[a]++;
            self(self, b);
        }
        path.push_back(a);
    };
    hierholzer(hierholzer, s);
    if (path.size() != e + 1) return {};
    reverse(path.begin(), path.end());
    return path;
}

4.15 hopcroft-karp
struct hopcroft_karp {
    const int inf = 1e9;
    int n, m, s;
    vector<int> r, l, d;
    vector<vector<int>> g;

```

```

hopcroft_karp(int n, int m): n(n), m(m), r(n, -1), l(m, -1),
d(n), g(n) {}

void edge(int a, int b) {
    g[a].push_back(b);
}

bool bfs() {
    s = inf;
    queue<int> q;
    for (int a = 0; a < n; ++a) {
        if (r[a] == -1) {
            d[a] = 0;
            q.push(a);
        } else {
            d[a] = inf;
        }
    }
    while (q.size()) {
        int a = q.front(); q.pop();
        if (d[a] >= s) continue;
        for (int b : g[a]) {
            if (l[b] == -1) {
                s = min(s, d[a] + 1);
            } else if (d[l[b]] == inf) {
                d[l[b]] = d[a] + 1;
                q.push(l[b]);
            }
        }
    }
    return s != inf;
}

bool dfs(int a) {
    if (a == -1) return true;
    for (int b : g[a]) {
        int h = (l[b] != -1) ? d[l[b]] : s;
        if (h == d[a] + 1) if (dfs(l[b])) {
            r[a] = b;
            l[b] = a;
            return true;
        }
    }
    d[a] = inf;
    return false;
}

int match() {
    int matching = 0;
    while (bfs()) {
        for (int a = 0; a < n; ++a) if (r[a] == -1 and dfs(a))
            matching++;
    }
    return matching;
}

vector<pair<int, int>> matches() {
    vector<pair<int, int>> m;
    for (int a = 0; a < n; ++a) if (r[a] != -1) m.emplace_back(a,
        r[a]);
    return m;
}
};

```

4.16 hungarian

```

// Hungarian Algorithm: minimum weight bipartite matching
// Time: O(N^2 * M) | Space: O(N + M)
// Note: computes min cost assignment | negate input matrix for max
pair<int, vector<int>> hungarian(vector<vector<int>> g) {
    int n = g.size(), m = g[0].size();
    bool transposed = false;
    if (n > m) {
        transposed = true;
        vector<vector<int>> t(m, vector<int>(n));
        for (int i = 0; i < n; ++i) {
            for (int j = 0; j < m; ++j) {
                t[j][i] = g[i][j];
            }
        }
        swap(n, m);
        swap(g, t);
    }

    vector<int> pot_row(n + 1), pot_col(m + 1);
    vector<int> match(m + 1), prev_col(m + 1);

    for (int i = 1; i <= n; ++i) {
        match[0] = i;
        int col = 0;
        vector<int> min_cost(m + 1, inf);
        vector<bool> v(m + 1);
        do {
            v[col] = true;
            int row = match[col];
            int delta = inf;
            int next_col = 0;
            for (int j = 1; j <= m; ++j) if (not v[j]) {
                int cost = g[row - 1][j - 1] - pot_row[row] -
                    pot_col[j];
                if (cost < min_cost[j]) {

```

```

                    min_cost[j] = cost;
                    prev_col[j] = col;
                }
            }
            if (min_cost[j] < delta) {
                delta = min_cost[j];
                next_col = j;
            }
        }
        for (int j = 0; j <= m; ++j) {
            if (v[j]) {
                pot_row[match[j]] += delta;
                pot_col[j] -= delta;
            } else {
                min_cost[j] -= delta;
            }
        }
        col = next_col;
    } while (match[col] != 0);
    do {
        int prev = prev_col[col];
        match[col] = match[prev];
        col = prev;
    } while (col != 0);
}

int cost = -pot_col[0];
vector<int> answer(n, -1);
for (int j = 1; j <= m; ++j) if (match[j] != 0) {
    answer[match[j] - 1] = j - 1;
}

if (transposed) {
    vector<int> real(m, -1);
    for (int i = 0; i < n; ++i) if (answer[i] != -1) {
        real[answer[i]] = i;
    }
    swap(answer, real);
}
return {cost, answer};
}

```

4.17 kuhn

```

// Kuhn's Algorithm: maximum bipartite matching
// Time: O(V * E) | Space: O(N + M)
// Note: connects left [0..n] to right [0..m] | match[right] returns
// left node or -1
vector<int> kuhn(int n, int m, vector<vector<int>> &g) {
    vector<int> match(m + 1, -1);
    int itr = 1;
    vector<int> v(n + 1);

    auto dfs = [&](auto &self, int a) -> bool {
        if (v[a] == itr) return false;
        v[a] = itr;
        for (int b : g[a]) if (match[b] == -1 or self(self,
            match[b])) {
            match[b] = a;
            return true;
        }
        return false;
    };

    for (int a = 0; a <= n; ++a, ++itr) {
        dfs(dfs, a);
    }
    return match;
}

```

4.18 maximum-clique

```

vector<int> maximum_clique(vector<vector<int>> &g) {
    int n = g.size();
    vector<int> h(n);
    for (int a = 0; a < n; ++a) {
        for (int b : g[a]) {
            h[a] |= 1LL << b;
        }
    }

    int size = 0, mask = 0;
    auto bron_kerbosch = [&](auto &self, int r, int p, int x) -> void
    {
        if (p == 0 and x == 0) {
            int s = __builtin_popcountll(r);
            if (s > size) {
                size = s;
                mask = r;
            }
            return;
        }
        int i = __builtin_ctzll(p | x);
        int c = p & ~h[i];
        while (c) {
            int k = __builtin_ctzll(c);
            int k_mask = 1LL << k;
            self(self, r | k_mask, p & h[k], x & h[k]);
            p ^= k_mask;
            x |= k_mask;
            c ^= k_mask;
        }
    };
}

```



```

int initial = (1LL << n) - 1;
bron_kerbosch(bron_kerbosch, 0, initial, 0);
vector<int> clique;
for (int i = 0; i < n; ++i) {
    if ((mask >> i) & 1) {
        clique.push_back(i);
    }
}
return clique;
}
}

```

4.19 binary-lifting

```

struct binary_lifting {
    int n, LOG;
    vector<int> d;
    vector<vector<int>> up;

    binary_lifting(vector<vector<int>> &g) {
        n = g.size();
        LOG = __bit_width(n) - 1;
        d.assign(n, -1);
        up.assign(n, vector<int>(LOG + 1));
        for (int i = 0; i < n; ++i) {
            if (d[i] == -1) {
                d[i] = 0;
                build(i, i, g);
            }
        }

        void build(int a, int p, vector<vector<int>> &g) {
            up[a][0] = p;
            for (int i = 1; i <= LOG; ++i) {
                up[a][i] = up[up[a][i - 1]][i - 1];
            }
            for (int b : g[a]) if (b != p) {
                d[b] = d[a] + 1;
                build(b, a, g);
            }
        }

        int kth(int a, int k) {
            k = min(k, d[a]);
            while (k > 0) {
                int i = __bit_width(k) - 1;
                a = up[a][i];
                k ^= 1 << i;
            }
            return a;
        }

        int lca(int a, int b) {
            if (d[a] < d[b]) swap(a, b);
            a = kth(a, d[a] - d[b]);
            if (a == b) return a;
            for (int i = LOG; i >= 0; --i) {
                if (up[a][i] != up[b][i]) {
                    a = up[a][i];
                    b = up[b][i];
                }
            }
            return up[a][0];
        }

        int dist(int a, int b) {
            return d[a] + d[b] - 2 * d[lca(a, b)];
        }

        int path(int a, int b, int k) {
            int p = lca(a, b);
            int l = d[a] - d[p];
            int r = d[b] - d[p];
            k = min(k, l + r);
            if (k <= l) return kth(a, k);
            else return kth(b, l + r - k);
        }
    };
}

```

4.20 centroid-decomposition

```

pair<vector<int>, vector<vector<int>>> centroid_decomposition(int r,
    vector<vector<int>> &g) {
    int n = g.size();
    vector<int> parent(n);
    vector<int> size(n);
    vector<bool> removed(n);
    vector<vector<int>> dist(n);
    auto dfs = [&](auto &self, int a, int p) -> int {
        size[a] = 1;
        for (int b : g[a]) if (b != p and not removed[b]) {
            size[a] += self(self, b, a);
        }
        return size[a];
    };
    auto centroid = [&](auto &self, int a, int p, int s) -> int {
        for (int b : g[a]) if ((b != p and not removed[b]) and
            size[b] > s / 2) {
            return self(self, b, a, s);
        }
        return a;
    };
}

```

```

};
auto distance = [&](auto &self, int a, int p, int d) -> void {
    dist[a].push_back(d);
    for (int b : g[a]) if (b != p and not removed[b]) {
        self(self, b, a, d + 1);
    }
};
auto build = [&](auto &self, int a, int p) -> void {
    int s = dfs(dfs, a, a);
    a = centroid(centroid, a, a, s);
    distance(distance, a, a, 0);
    parent[a] = p;
    removed[a] = true;
    for (int b : g[a]) if (not removed[b]) {
        self(self, b, a);
    }
};
build(build, r, -1);
for (auto &path : dist) reverse(path.begin(), path.end());
return {parent, dist};
}

```

4.21 heavy-light-decomposition

```

struct hld {
    int n;
    bool e;
    segment_tree s;
    vector<int> p, d, h, id;

    hld(vector<vector<int>> &g, bool e): n(g.size()), e(e), s(n) {
        p.assign(n, -1);
        d.assign(n, 0);
        h.assign(n, 0);
        id.assign(n, 0);
        int t = 0;
        for (int i = 0; i < n; ++i) {
            if (p[i] == -1) {
                p[i] = i;
                d[i] = 0;
                dfs(i, g);
                h[i] = i;
                decompose(i, t, g);
            }
        }

        int dfs(int a, vector<vector<int>> &g) {
            int size = 1, max = 0;
            for (int &b : g[a]) if (b != p[a]) {
                p[b] = a;
                d[b] = d[a] + 1;
                int curr = dfs(b, g);
                if (curr > max) {
                    max = curr;
                    swap(g[a][0], b);
                }
                size += curr;
            }
            return size;
        }

        void decompose(int a, int &t, vector<vector<int>> &g) {
            id[a] = t++;
            for (int b : g[a]) if (b != p[a]) {
                h[b] = b;
                if (b == g[a][0]) h[b] = h[a];
                decompose(b, t, g);
            }
        }

        int lca(int a, int b) {
            while (h[a] != h[b]) {
                if (d[h[a]] < d[h[b]]) swap(a, b);
                a = p[h[a]];
            }
            if (d[a] > d[b]) swap(a, b);
            return a;
        }

        int query(int a, int b) {
            int r = s.neutral;
            while (h[a] != h[b]) {
                while (d[h[a]] < d[h[b]]) swap(a, b);
                r = s.merge(r, s.query(id[h[a]], id[a]));
                a = p[h[a]];
            }
            if (d[a] > d[b]) swap(a, b);
            if (e and a == b) return r;
            r = s.merge(r, s.query(id[a] + e, id[b]));
            return r;
        }

        void update(int a, int x) {
            s.update(id[a], x);
        }
    };
}

```

4.22 lazy-heavy-light-decomposition

```

struct lazy_hld {

```

```

int n;
bool e;
lazy_segment_tree s;
vector<int> p, d, h, id;

lazy_hld(vector<vector<int>> &g, bool e): n(g.size()), e(e), s(n)
{
    p.assign(n, -1);
    d.assign(n, 0);
    h.assign(n, 0);
    id.assign(n, 0);
    int t = 0;
    for (int i = 0; i < n; ++i) {
        if (p[i] == -1) {
            p[i] = i;
            d[i] = 0;
            dfs(i, g);
            h[i] = i;
            decompose(i, t, g);
        }
    }
}

int dfs(int a, vector<vector<int>> &g) {
    int size = 1, max = 0;
    for (int &b : g[a]) if (b != p[a]) {
        p[b] = a;
        d[b] = d[a] + 1;
        int curr = dfs(b, g);
        if (curr > max) {
            max = curr;
            swap(g[a][0], b);
        }
        size += curr;
    }
    return size;
}

void decompose(int a, int &t, vector<vector<int>> &g) {
    id[a] = t++;
    for (int b : g[a]) if (b != p[a]) {
        h[b] = b;
        if (b == g[a][0]) h[b] = h[a];
        decompose(b, t, g);
    }
}

int lca(int a, int b) {
    while (h[a] != h[b]) {
        if (d[h[a]] < d[h[b]]) swap(a, b);
        a = p[h[a]];
    }
    if (d[a] > d[b]) swap(a, b);
    return a;
}

int query(int a, int b) {
    int r = s.neutral;
    while (h[a] != h[b]) {
        if (d[h[a]] < d[h[b]]) swap(a, b);
        r = s.merge(r, s.query(id[h[a]], id[a]));
        a = p[h[a]];
    }
    if (d[a] > d[b]) swap(a, b);
    if (e and a == b) return r;
    r = s.merge(r, s.query(id[a] + e, id[b]));
    return r;
}

void update(int a, int b, int x) {
    while (h[a] != h[b]) {
        if (d[h[a]] < d[h[b]]) swap(a, b);
        s.update(id[h[a]], id[a], x);
        a = p[h[a]];
    }
    if (d[a] > d[b]) swap(a, b);
    if (e and a == b) return;
    s.update(id[a] + e, id[b], x);
}
};

```

4.23 leaf-peeling

```

// Leaf Peeling: finds the center(s) and diameter of an undirected
// tree
// Time: O(N) | Space: O(N)
// Note: returns {diameter, centers} | 'i' sets 0-based or 1-based
// indexing
pair<int, vector<int>> leaf_peeling(int i, vector<vector<int>> &g) {
    int n = g.size() - i;
    if (n == 1) return {0, {i}};
    vector<int> d(g.size());
    queue<int> q;
    for (int a = i; a < g.size(); ++a) {
        d[a] = g[a].size();
        if (d[a] == 1) q.push(a);
    }
    int r = n, l = 0;
    while (r > 2) {
        int s = q.size();
        r -= s;
    }
}

```

```

l++;
while (s-- > 0) {
    int a = q.front(); q.pop();
    for (int b : g[a]) {
        if (--d[b] == 1) q.push(b);
    }
}
vector<int> c;
while (q.size()) c.push_back(q.front()); q.pop();
return {2 * l + (c.size() == 2), c};
}

```

4.24 reversible-heavy-light-decomposition

```

struct reversible_hld {
    int n;
    bool e;
    reversible_segment_tree s, t;
    vector<int> p, d, h, id;

    reversible_hld(vector<vector<int>> &g, bool e): n(g.size()),
        e(e), s(n, false), t(n, true) {
        p.assign(n, -1);
        d.assign(n, 0);
        h.assign(n, 0);
        id.assign(n, 0);
        int k = 0;
        for (int i = 0; i < n; ++i) {
            if (p[i] == -1) {
                p[i] = i;
                d[i] = 0;
                dfs(i, g);
                h[i] = i;
                decompose(i, k, g);
            }
        }
    }

    int dfs(int a, vector<vector<int>> &g) {
        int size = 1, max = 0;
        for (int &b : g[a]) if (b != p[a]) {
            p[b] = a;
            d[b] = d[a] + 1;
            int curr = dfs(b, g);
            if (curr > max) {
                max = curr;
                swap(g[a][0], b);
            }
            size += curr;
        }
        return size;
    }

    void decompose(int a, int &k, vector<vector<int>> &g) {
        id[a] = k++;
        for (int b : g[a]) if (b != p[a]) {
            h[b] = b;
            if (b == g[a][0]) h[b] = h[a];
            decompose(b, k, g);
        }
    }

    int lca(int a, int b) {
        while (h[a] != h[b]) {
            if (d[h[a]] < d[h[b]]) swap(a, b);
            a = p[h[a]];
        }
        if (d[a] > d[b]) swap(a, b);
        return a;
    }

    int query(int a, int b) {
        int l = s.neutral;
        int r = s.neutral;
        while (h[a] != h[b]) {
            if (d[h[a]] > d[h[b]]) {
                l = t.merge(l, t.query(id[h[a]], id[a]));
                a = p[h[a]];
            } else {
                r = s.merge(s.query(id[h[b]], id[b]), r);
                b = p[h[b]];
            }
        }
        if (e and a == b) return s.merge(l, r);
        if (d[a] > d[b]) {
            l = t.merge(l, t.query(id[b] + e, id[a]));
        } else {
            r = s.merge(s.query(id[a] + e, id[b]), r);
        }
        return s.merge(l, r);
    }

    void update(int a, int x) {
        s.update(id[a], x);
        t.update(id[a], x);
    }
};

```

4.25 two-sat

```

// 2-SAT: Solves Boolean Satisfiability for 2-CNF Formulas
// Time: Build  $O(N + M)$  | Space:  $O(N + M)$ 
// Note: Uses Tarjan's SCC | 0-indexed variables
struct two_sat {
    int n;
    vector<vector<int>> g;

    two_sat(int n): n(n), g(2 * n) {}

    // A or B
    void disjunction(int a, bool n_a, int b, bool n_b) {
        a = (2 * a) ^ n_a;
        b = (2 * b) ^ n_b;
        g[a ^ 1].push_back(b);
        g[b ^ 1].push_back(a);
    }

    // A and B
    void conjunction(int a, bool n_a, int b, bool n_b) {
        disjunction(a, n_a, a, n_a);
        disjunction(b, n_b, b, n_b);
    }

    // A -> B
    void implication(int a, bool n_a, int b, bool n_b) {
        disjunction(a, !n_a, b, n_b);
    }

    // A == B
    void equal(int a, bool n_a, int b, bool n_b) {
        implication(a, n_a, b, n_b);
        implication(b, n_b, a, n_a);
    }

    // A != B / A xor B
    void not_equal(int a, bool n_a, int b, bool n_b) {
        disjunction(a, n_a, b, n_b);
        disjunction(a, !n_a, b, !n_b);
    }

    // Solves the 2-SAT and returns a valid assignment, or empty if
    // unsatisfiable
    vector<bool> build() {
        vector<vector<int>> components = tarjan(0, g);
        int m = components.size();
        vector<int> id(2 * n);
        for (int i = 0; i < m; ++i) {
            for (int a : components[i]) {
                id[a] = i;
            }
        }
        vector<bool> assignment(n);
        for (int i = 0; i < n; ++i) {
            if (id[2 * i] == id[(2 * i) ^ 1]) return {};
            assignment[i] = id[2 * i] > id[(2 * i) ^ 1];
        }
        return assignment;
    }
};

```

5 math

5.1 fast-fourier-transform

```

const float pi = acos(-1);
const int max_len = 1 << 20;

array<complex<float>, max_len / 2> root, inv_root;

void roots() {
    root[0] = {1, 0};
    inv_root[0] = {1, 0};

    float ang = 2 * pi / max_len;
    int half = max_len / 2;
    for (int i = 1; i < half; i *= 2) {
        complex<float> step(cos(ang * i), sin(ang * i));
        for (int j = 0; j < i; ++j) {
            root[i + j] = root[j] * step;
        }
    }
    for (int i = 1; i < half; ++i) {
        inv_root[i] = conj(root[i]);
    }
}

void fft(vector<complex<float>> &p, bool inv) {
    int n = p.size();
    if (not inv) {
        for (int len = n; len >= 2; len /= 2) {
            int half = len / 2;
            int step = max_len / len;
            for (int i = 0; i < n; i += len) {
                for (int j = 0; j < half; ++j) {
                    auto w = root[step * j];
                    auto x = p[i + j];
                    auto y = p[i + j + half];
                    p[i + j] = x + y;

```

```

                    p[i + j + half] = (x - y) * w;
                }
            }
        }
    } else {
        for (int len = 2; len <= n; len *= 2) {
            int half = len / 2;
            int step = max_len / len;
            for (int i = 0; i < n; i += len) {
                for (int j = 0; j < half; ++j) {
                    auto w = inv_root[step * j];
                    auto x = p[i + j];
                    auto y = p[i + j + half] * w;
                    p[i + j] = x + y;
                    p[i + j + half] = x - y;
                }
            }
        }
        for (auto &x : p) x /= n;
    }
}

vector<int> convolution(vector<int> &a, vector<int> &b) {
    int n = a.size() + b.size() - 1;
    int m = 1;
    while (m < n) m *= 2;

    vector<complex<float>> fa(a.begin(), a.end());
    vector<complex<float>> fb(b.begin(), b.end());
    fa.resize(m);
    fb.resize(m);
    fft(fa, false);
    fft(fb, false);

    for (int i = 0; i < m; ++i) fa[i] *= fb[i];
    fft(fa, true);

    vector<int> c(n);
    for (int i = 0; i < n; ++i) c[i] = round(fa[i].real());
    return c;
}

```

5.2 fast-walsh-hadamard-transform

```

// FWHT / Bitwise Convolution:  $C[k] = \sum(A[i] * B[j])$  for  $i$  (OPR)  $j = k$ 
// Time:  $O(N \log N)$  | Space:  $O(N)$ 
// Operation: 'A' (AND), 'O' (OR), 'X' (XOR)
void fwht(vector<int> &p, int sign, char opr) {
    int n = p.size();

    for (int half = 1; half < n; half *= 2) {
        for (int i = 0; i < n; i += 2 * half) {
            for (int j = 0; j < half; ++j) {
                int x = p[i + j];
                int y = p[(i + j) + half];
                if (opr == 'A') {
                    p[i + j] = ((x + sign * y) + mod) % mod;
                } else if (opr == 'O') {
                    p[(i + j) + half] = ((sign * x + y) + mod) % mod;
                } else if (opr == 'X') {
                    p[i + j] = (x + y) % mod;
                    p[(i + j) + half] = ((x - y) + mod) % mod;
                }
            }
        }
    }

    if (opr == 'X' and sign == -1) {
        int inv_n = mod_pow(n, mod - 2);
        for (int &i : p) i = (i * inv_n) % mod;
    }
}

vector<int> convolution(vector<int> a, vector<int> b, char opr) {
    int n = max(a.size(), b.size());
    int m = 1;
    while (m < n) m *= 2;

    a.resize(m);
    b.resize(m);
    fwht(a, +1, opr);
    fwht(b, +1, opr);

    for (int i = 0; i < m; ++i) a[i] = (a[i] * b[i]) % mod;
    fwht(a, -1, opr);

    return a;
}

```

5.3 modulo-transform-technique

```

vector<int> roots(int half, int ang, int mod) {
    vector<int> w(half);
    w[0] = 1;
    for (int i = 1; i < half; ++i) w[i] = (ang * w[i - 1]) % mod;
    return w;
}

void ntt(vector<int> &p, bool inv, int g, int mod) {
    int n = p.size();

```

```

if (not inv) {
    for (int len = n; len >= 2; len /= 2) {
        int half = len / 2;
        int ang = mpow(g, (mod - 1) / len, mod);
        vector<int> w = roots(half, ang, mod);
        for (int i = 0; i < n; i += len) {
            for (int j = 0; j < half; ++j) {
                int x = p[i + j];
                int y = p[i + j + half];
                p[i + j] = (x + y) % mod;
                p[i + j + half] = ((x - y + mod) % mod) * w[j]
                    % mod;
            }
        }
    }
} else {
    for (int len = 2; len <= n; len *= 2) {
        int half = len / 2;
        int ang = mpow(g, (mod - 1) / len, mod);
        ang = mpow(ang, mod - 2, mod);
        vector<int> w = roots(half, ang, mod);
        for (int i = 0; i < n; i += len) {
            for (int j = 0; j < half; ++j) {
                int x = p[i + j];
                int y = (p[i + j + half] * w[j]) % mod;
                p[i + j] = (x + y) % mod;
                p[i + j + half] = (x - y + mod) % mod;
            }
        }
    }
    int inv_n = mpow(n, mod - 2, mod);
    for (int &x : p) x = (x * inv_n) % mod;
}

vector<int> convolution(vector<int> a, vector<int> b, int g = 3, int
    mod = 998244353) {
    int n = a.size() + b.size() - 1;
    int m = 1;
    while (m < n) m *= 2;

    a.resize(m);
    b.resize(m);
    ntt(a, false, g, mod);
    ntt(b, false, g, mod);

    for (int i = 0; i < m; ++i) a[i] = (a[i] * b[i]) % mod;
    ntt(a, true, g, mod);
    a.resize(n);

    return a;
}

vector<int> mtt(vector<int> &a, vector<int> &b, int mod) {
    vector<int> mods = {998244353, 1004535809, 469762049};

    vector<int> c1 = convolution(a, b, 3, mods[0]);
    vector<int> c2 = convolution(a, b, 3, mods[1]);
    vector<int> c3 = convolution(a, b, 3, mods[2]);

    int n = c1.size();
    vector<int> c(n);
    for (int i = 0; i < n; ++i) {
        vector<int> rems = {c1[i], c2[i], c3[i]};
        int128 x = chinese_remainder_theorem(rems, mods);
        c[i] = x % mod;
    }
    return c;
}

```

5.4 number-theoretic-transform

```

const int g = 3;
const int mod = 998244353;
const int max_len = 1 << 20;

array<int, max_len / 2> root, inv_root;

void roots() {
    root[0] = 1;
    inv_root[0] = 1;

    int ang = mpow(g, (mod - 1) / max_len);
    int inv_ang = mpow(ang, mod - 2);

    int half = max_len / 2;
    for (int i = 1; i < half; ++i) {
        root[i] = (ang * root[i - 1]) % mod;
        inv_root[i] = (inv_ang * inv_root[i - 1]) % mod;
    }
}

void ntt(vector<int> &p, bool inv) {
    int n = p.size();
    if (not inv) {
        for (int len = n; len >= 2; len /= 2) {
            int half = len / 2;
            int step = max_len / len;
            for (int i = 0; i < n; i += len) {
                for (int j = 0; j < half; ++j) {
                    int w = root[step * j];

```

```

                int x = p[i + j];
                int y = p[i + j + half];
                p[i + j] = (x + y) % mod;
                p[i + j + half] = ((x - y + mod) % mod) * w %
                    mod;
            }
        }
    } else {
        for (int len = 2; len <= n; len *= 2) {
            int half = len / 2;
            int step = max_len / len;
            for (int i = 0; i < n; i += len) {
                for (int j = 0; j < half; ++j) {
                    int w = inv_root[step * j];
                    int x = p[i + j];
                    int y = (p[i + j + half] * w) % mod;
                    p[i + j] = (x + y) % mod;
                    p[i + j + half] = (x - y + mod) % mod;
                }
            }
        }
        int inv_n = mpow(n, mod - 2);
        for (int &x : p) x = (x * inv_n) % mod;
    }
}

vector<int> convolution(vector<int> a, vector<int> b) {
    int n = a.size() + b.size() - 1;
    int m = 1;
    while (m < n) m *= 2;

    a.resize(m);
    b.resize(m);
    ntt(a, false);
    ntt(b, false);

    for (int i = 0; i < m; ++i) a[i] = (a[i] * b[i]) % mod;
    ntt(a, true);
    a.resize(n);

    return a;
}

```

5.5 gaussian-elimination

```

// Gaussian Elimination: solves systems of linear equations (modulo
    arithmetic)
// Time: O(N * M^2) | Space: O(M) auxiliary
// Note: returns {(0: none, 1: unique, 2: infinite), solution}
pair<int, vector<int>> gaussian_elimination(matrix &a, const int mod)
{
    int n = a.size(), m = a[0].size() - 1;
    vector<int> p(m, -1);
    int r, c;
    for (r = 0, c = 0; r < n and c < m; ++c) {
        int k = r;
        for (int i = r + 1; i < n and a[k][c] == 0; ++i) k = i;
        if (a[k][c] == 0) continue;

        swap(a[r], a[k]);
        p[r] = c;

        int inv = mod_pow(a[r][c], mod - 2, mod);
        for (int j = c; j <= m; ++j) {
            a[r][j] = (a[r][j] * inv) % mod;
        }

        for (int i = 0; i < n; ++i) if (i != r and a[i][c] != 0) {
            int f = mod - a[i][c];
            for (int j = c; j <= m; ++j) {
                a[i][j] = (a[i][j] + f * a[r][j]) % mod;
            }
        }
        ++r;
    }
    for (int i = r; i < n; ++i) if (a[i][m] != 0) {
        return {0, {}};
    }
    vector<int> x(m);
    for (int i = 0; i < r; ++i) {
        x[p[i]] = a[i][m];
    }
    return {(r == m) ? 1 : 2, x};
}

// Gaussian Elimination: solves systems of linear equations
    (floating-point with partial pivoting)
// Time: O(N * M^2) | Space: O(M) auxiliary
// Note: returns {(0: none, 1: unique, 2: infinite), solution}. Uses
    'eps' for numerical stability.
pair<int, vector<float>> gaussian_elimination(matrix &a, const float
    eps) {
    int n = a.size(), m = a[0].size() - 1;
    vector<int> p(m, -1);
    int r, c;
    for (r = 0, c = 0; r < n and c < m; ++c) {
        int k = r;
        for (int i = r + 1; i < n; ++i) if (abs(a[i][c]) >
            abs(a[k][c])) k = i;
        if (abs(a[k][c]) < eps) continue;

```

```

swap(a[r], a[k]);
p[r] = c;

float d = a[r][c];
for (int j = c; j <= m; ++j) {
    a[r][j] /= d;
}

for (int i = 0; i < n; ++i) if (i != r and abs(a[i][c]) >
    eps) {
    float f = a[i][c];
    for (int j = c; j <= m; ++j) {
        a[i][j] -= f * a[r][j];
    }
}
++r;

for (int i = r; i < n; ++i) if (abs(a[i][m]) > eps) {
    return {0, {}};
}
vector<float> x(m);
for (int i = 0; i < r; ++i) {
    x[p[i]] = a[i][m];
}
return {(r == m) ? 1 : 2, x};
}

```

5.6 josephus

```

// Josephus: finds survivor index eliminating every k-th person
// Time: O(N) | Space: O(N)
// Note: returns 0-based index
int josephus(int n, int k) {
    if (n == 1) return 0;
    return (josephus(n - 1, k) + k) % n;
}

```

5.7 berlekamp-massey

```

// Berlekamp-Massey: finds the shortest linear recurrence of a
// sequence
// Time: O(N^2) where N is sequence size | Space: O(N)
// Note: requires at least 2L terms to correctly find a recurrence of
// length L
vector<int> berlekamp_massey(const vector<int> &seq) {
    vector<int> old = {1}, curr = {1};
    int prev = 1, len = 0, shift = 1;
    for (int i = 0; i < seq.size(); ++i) {
        int delta = 0;
        for (int j = 0; j <= len; ++j) {
            delta = (delta + curr[j] * seq[i - j]) % mod;
        }
        if (delta == 0) {
            shift++;
            continue;
        }
        vector<int> temp = curr;
        int mult = (mod - (delta * power(prev, mod - 2)) % mod) % mod;
        if (curr.size() <= old.size() + shift) {
            curr.resize(old.size() + shift + 1, 0);
        }
        for (int j = 0; j < old.size(); ++j) {
            curr[j + shift] = (curr[j + shift] + mult * old[j]) % mod;
        }
        if (2 * len <= i) {
            len = i + 1 - len;
            old = temp;
            prev = delta;
            shift = 1;
        } else {
            shift++;
        }
    }
    vector<int> coef(len);
    for (int i = 1; i <= len; ++i) {
        coef[i - 1] = (mod - curr[i]) % mod;
    }
    return coef;
}

```

5.8 kth-term

```

// Linear Recurrence K-th Term: computes the K-th term of a sequence
// Time: O(L^2 log K) where L is recurrence length | Space: O(L)
// Note: 0-indexed | the sequence must have at least L terms
int kth_term(int k, const vector<int> &seq, const vector<int> &coef) {
    if (k < seq.size()) return seq[k];
    if (coef.empty()) return 0;

    int n = coef.size();

    auto combine = [&](const vector<int> &a, const vector<int> &b) ->
        vector<int> {
            vector<int> r(2 * n);
            for (int i = 0; i < n; ++i) {
                for (int j = 0; j < n; ++j) {
                    r[i + j] = (r[i + j] + a[i] * b[j]) % mod;
                }
            }
            for (int i = 2 * n - 2; i >= n; --i) {

```

```

                for (int j = 0; j < n; ++j) {
                    r[i - j - 1] = (r[i - j - 1] + r[i] * coef[j]) % mod;
                }
            }
            r.resize(n);
            return r;
        };

    vector<int> pol(n), exp(n);
    pol[0] = 1;
    if (n > 1) exp[1] = 1;
    else exp[0] = coef[0];

    while (k != 0) {
        if (k & 1) pol = combine(pol, exp);
        exp = combine(exp, exp);
        k >>= 1;
    }

    int ans = 0;
    for (int i = 0; i < n; ++i) {
        ans = (ans + pol[i] * seq[i]) % mod;
    }
    return ans;
}

```

5.9 matrix

```

using matrix = vector<vector<int>>;

matrix operator*(const matrix &a, const matrix &b) {
    int n = a.size();
    int m = b[0].size();
    int h = b.size();
    matrix r(n, vector<int>(m));
    for (int i = 0; i < n; ++i) {
        for (int k = 0; k < h; ++k) {
            for (int j = 0; j < m; ++j) {
                r[i][j] = (r[i][j] + a[i][k] * b[k][j]) % mod;
            }
        }
    }
    return r;
}

matrix operator^(matrix a, int b) {
    int n = a.size();
    matrix r(n, vector<int>(n));
    for (int i = 0; i < n; ++i) r[i][i] = 1;
    while (b != 0) {
        if (b & 1) r = r * a;
        a = a * a;
        b >>= 1;
    }
    return r;
}

```

5.10 min-plus-matrix

```

using matrix = vector<vector<int>>;

matrix operator*(const matrix &a, const matrix &b) {
    int n = a.size();
    int m = b[0].size();
    int h = b.size();
    matrix r(n, vector<int>(m, inf));
    for (int i = 0; i < n; ++i) {
        for (int k = 0; k < h; ++k) {
            for (int j = 0; j < m; ++j) {
                r[i][j] = min(r[i][j], a[i][k] + b[k][j]);
            }
        }
    }
    return r;
}

matrix operator^(matrix a, int b) {
    int n = a.size();
    matrix r(n, vector<int>(n, inf));
    for (int i = 0; i < n; ++i) r[i][i] = 0;
    while (b != 0) {
        if (b & 1) r = r * a;
        a = a * a;
        b >>= 1;
    }
    return r;
}

```

5.11 nim

```

bool nim(vector<int> &piles, vector<int> &moves) {
    int n = *max(piles.begin(), piles.end());
    vector<int> grundy(n + 1);

    for (int i = 1; i <= n; ++i) {
        set<int> reach;
        for (int move : moves) if (move <= i) {
            reach.insert(grundy[i - move]);
        }
        int mex = 0;

```

```

    while (reach.contains(mex)) mex++;
    grundy[i] = mex;
}

int answer = 0;
for (int pile : piles) {
    answer ^= grundy[pile];
}
return answer != 0;
}

bool misere(vector<int> &piles, vector<int> &moves) {
    int n = *max(piles.begin(), piles.end());
    vector<int> grundy(n + 1);

    for (int i = 1; i <= n; ++i) {
        set<int> reach;
        for (int move : moves) if (move <= i) {
            reach.insert(grundy[i - move]);
        }

        int mex = 0;
        while (reach.contains(mex)) mex++;
        grundy[i] = mex;
    }

    bool all_one = true;
    int answer = 0;
    for (int pile : piles) {
        all_one &= grundy[pile] == 1;
        answer ^= grundy[pile];
    }
    return all_one ? (piles.size() & 1) == 0 : answer != 0;
}

```

5.12 xor-basis

```

// XOR Basis: linear basis for subset XOR representation and query
// operations
// Time: insert/contains/max/kth O(log) | Space: O(log) where log is
// the maximum bit length (30)
// Note: kth(k) returns k-th smallest distinct XOR | operator[k]
// returns k-th smallest among all 2^n subsets (with duplicates)
struct xor_basis {
    static constexpr int log = 30;
    int n = 0, s = 0;
    array<int, log> basis = {};

    void insert(int x) {
        for (int i = log - 1; i >= 0; --i) if ((x >> i) & 1) {
            if (basis[i] == 0) {
                for (int j = i - 1; j >= 0; --j) if ((x >> j) & 1) {
                    x ^= basis[j];
                }
                basis[i] = x;
                s += 1;
                for (int j = log - 1; j > i; --j) if ((basis[j] >> i)
                    & 1) {
                    basis[j] ^= x;
                }
                break;
            } else {
                x ^= basis[i];
            }
        }
        n += 1;
    }

    bool contains(int x) {
        for (int i = log - 1; i >= 0; --i) if ((x >> i) & 1) {
            x ^= basis[i];
        }
        return x == 0;
    }

    int max() {
        int x = 0;
        for (int i = log - 1; i >= 0; --i) if (((x >> i) & 1) == 0) {
            x ^= basis[i];
        }
        return x;
    }

    int kth(int k) {
        if ((k >> s) != 0) return -1;
        int x = 0;
        for (int i = 0; i < log; ++i) if (basis[i] != 0) {
            if (k & 1) x ^= basis[i];
            k >>= 1;
        }
        return x;
    }

    int operator[](int k) {
        if (n - s > 60) return 0;
        return kth(k >> (n - s));
    }
};

```

6 number-theory

6.1 harmonic-lemma

```

// Harmonic Lemma: Quotient Blocks in a specific range [l, r]
// Time: O(sqrt(N)) | Space: O(1)
// Note: Iterates over ranges [i, j] inside [l, r] where floor(n / k)
// is exactly q
int harmonic_lemma(int n, int l, int r) {
    int s = 0;
    for (int i = l, j; i <= r; i = j + 1) {
        int q = n / i;
        if (q == 0) {
            j = r;
        } else {
            j = min(r, n / q);
        }
        s += q * (j - i + 1);
    }
    return s;
}

```

6.2 mobius

```

int mobius(int n) {
    int mu = 1;
    for (int i = 2; i * i <= n; ++i) if (n % i == 0) {
        n /= i;
        if (n % i == 0) return 0;
        mu *= -1;
    }
    if (n > 1) mu *= -1;
    return mu;
}

```

```

vector<int> mobius_sieve(int n) {
    vector<int> mu(n + 1);
    vector<bool> is_prime(n + 1, true);
    vector<int> primes;
    mu[1] = 1;
    is_prime[1] = false;
    for (int i = 2; i <= n; ++i) {
        if (is_prime[i]) {
            primes.push_back(i);
            mu[i] = -1;
        }
        for (int p : primes) {
            if (i * p > n) break;
            is_prime[i * p] = false;
            if (i % p == 0) {
                mu[i * p] = 0;
                break;
            } else {
                mu[i * p] = -mu[i];
            }
        }
    }
    return mu;
}

```

6.3 chinese-remainder-theorem

```

int128 chinese_remainder_theorem(const vector<int> &rem, const
vector<int> &mod) {
    int128 r = rem[0], m = mod[0];
    for (int i = 1; i < rem.size(); ++i) {
        int g = gcd(static_cast<int>(m % mod[i]), mod[i]), h = mod[i]
            / g;
        if ((rem[i] - r) % g != 0) return -1;

        auto [x, _] = extended_gcd(static_cast<int>(m / g % h), h);

        r += m * (((rem[i] - r) / g) * x) % h + h) % h;
        m *= h;
    }
    return r;
}

```

6.4 extended-gcd

```

// Extended GCD: finds coefficients x and y such that ax + by =
// gcd(a, b)
// Time: O(log(min(a, b))) | Space: O(log(min(a, b)))
// Note: returns [x0, y0] | all valid pairs are: x = x0 + k*(b/gcd)
// and y = y0 - k*(a/gcd)
pair<int, int> extended_gcd(int a, int b) {
    if (b == 0) return {1, 0};

    auto [x, y] = extended_gcd(b, a % b);
    return {y, x - (a / b) * y};
}

```

6.5 mod-log

```

int mod_log(int a, int b, int mod) {
    if (a == 0) {
        if (b == 0) return 1 % mod;
        return -1;
    }

    int k = 1, h = 0;

```

```

for (int g = gcd(a, mod); g != 1; g = gcd(a, mod), ++h) {
    if (k == b) return h;
    if (b % g) return -1;

    b /= g, mod /= g;
    k = (k * (a / g)) % mod;
}

int n = ceil(sqrt(mod)), s = 1;
for (int i = 1; i <= n; ++i) {
    s = (s * a) % mod;
}

map<int, int> f;
for (int q = 0; q <= n; ++q) {
    f[b] = q;
    b = (b * a) % mod;
}

a = (k * s) % mod;
for (int p = 1; p <= n; ++p) {
    if (f.contains(a)) {
        return (n * p - f[a]) + h;
    }
    a = (a * s) % mod;
}
return -1;
}

```

6.6 mod-pow

```

int mod_pow(int a, int e, const int mod) {
    int r = 1;
    while (e != 0) {
        if (e & 1) r = (r * a) % mod;
        a = (a * a) % mod;
        e >>= 1;
    }
    return r;
}

```

6.7 multi-extended-gcd

```

vector<int> multi_extended_gcd(vector<int> &a) {
    int n = a.size(), g = a[0];
    vector<int> c(n, 1), h(n - 1);

    for (int i = 1; i < n; ++i) {
        auto [x, y] = extended_gcd(g, a[i]);
        g = gcd(g, a[i]);
        h[i - 1] = x;
        c[i] = y;
    }

    int p = 1;
    for (int i = n - 1; i > 0; --i) {
        c[i] *= p;
        p *= h[i - 1];
    }
    c[0] *= p;
    return c;
}

```

6.8 linear-sieve

```

vector<int> linear_sieve(int n) {
    vector<int> p, spf(n + 1);
    for (int i = 2; i <= n; ++i) {
        if (spf[i] == 0) {
            spf[i] = i;
            p.push_back(i);
        }
        for (int j : p) {
            if (j > spf[i] or i * j > n) break;
            spf[i * j] = j;
        }
    }
    return p;
}

```

6.9 miller-rabin

```

int mod_pow(int128 a, int e, const int mod) {
    int128 r = 1;
    while (e != 0) {
        if (e & 1) r = (r * a) % mod;
        a = (a * a) % mod;
        e >>= 1;
    }
    return r;
}

bool composite(int n, int a, int s, int d) {
    int128 x = mod_pow(a, d, n);
    if (x == 1 or x == n - 1) return false;
    for (int i = 1; i < s; ++i) {
        x = (x * x) % n;
        if (x == n - 1) return false;
        if (x == 1) return true;
    }
    return true;
}

```

```

}

bool prime(int n) {
    if (n <= 1) return false;
    static const int P[] = {2, 3, 5, 13, 19, 73, 193, 407521,
        299210837};
    for (int p : P) {
        if (n == p) return true;
        if (n % p == 0) return false;
    }
    int s = __builtin_ctzll(n - 1);
    int d = (n - 1) >> s;
    static const int A[] = {2, 325, 9375, 28178, 450775, 9780504,
        1795265022};
    for (int a : A) {
        if (composite(n, a, s, d)) return false;
    }
    return true;
}

```

6.10 segmented-sieve

```

vector<int> segmented_sieve(int l, int r) {
    vector<int> primes = linear_sieve(sqrt(r));
    offset_array<char> is_prime(l, r + 1, true);
    for (int p : primes) {
        for (int i = max(p * ((l + p - 1) / p), p * p); i <= r; i += p) {
            is_prime[i] = false;
        }
    }
    primes.clear();
    for (int i = max(2, l); i <= r; ++i) {
        if (is_prime[i]) {
            primes.push_back(i);
        }
    }
    return primes;
}

```

6.11 totient

```

int totient(int n) {
    int phi = n;
    for (int i = 2; i * i <= n; ++i) if (n % i == 0) {
        phi -= phi / i;
        while (n % i == 0) n /= i;
    }
    if (n > 1) phi -= phi / n;
    return phi;
}

vector<int> totient_sieve(int n) {
    vector<int> phi(n + 1);
    for (int i = 1; i <= n; ++i) {
        phi[i] = i;
    }
    for (int i = 2; i <= n; ++i) if (phi[i] == i) {
        for (int j = i; j <= n; j += i) {
            phi[j] -= phi[j] / i;
        }
    }
    return phi;
}

```

7 search

7.1 ternary-search

```

int ternary_search(int low, int high, bool max) {
    while (high - low > 3) {
        int mid_low = low + (high - low) / 3;
        int mid_high = high - (high - low) / 3;

        if (max == (value(mid_low) > value(mid_high))) {
            high = mid_high;
        } else {
            low = mid_low;
        }
    }

    int answer = low;
    for (int i = low + 1; i <= high; ++i) {
        if (max == (value(i) > value(answer))) {
            answer = i;
        }
    }
    return answer;
}

float ternary_search(float low, float high, bool max, float eps = 1e-6) {
    while (high - low > eps) {
        float mid_low = low + (high - low) / 3;
        float mid_high = high - (high - low) / 3;

        if (max == (value(mid_low) > value(mid_high))) {
            high = mid_high;
        }
    }
}

```

```

    } else {
        low = mid_low;
    }
}
return (low + high) / 2;
}

```

8 sheet

9 square-root

9.1 decomposition

```

struct decomposition {
    int n, m;
    vector<int> a, b;

    decomposition(vector<int> &base, float f = 1.0) {
        n = base.size();
        m = clamp<int>(f * sqrt(n), 1, n);
        a.resize(n);
        b.resize((n + (m - 1)) / m);
        for (int i = 0; i < n; ++i) {
            a[i] = base[i];
            b[i / m] += a[i];
        }
    }

    void update(int i, int x) {
        a[i] = x;
        int b_i = i / m;
        b[b_i] = 0;
        int l = b_i * m;
        int r = min(n, (b_i + 1) * m) - 1;
        for (int k = l; k <= r; ++k) {
            b[b_i] += a[k];
        }
    }

    int query(int l, int r) {
        int s = 0;
        int b_l = l / m;
        int b_r = r / m;
        if (b_l == b_r) {
            for (int i = l; i <= r; ++i) {
                s += a[i];
            }
        } else {
            for (int i = l; i < (b_l + 1) * m; ++i) {
                s += a[i];
            }
            for (int i = b_l + 1; i < b_r; ++i) {
                s += b[i];
            }
            for (int i = b_r * m; i <= r; ++i) {
                s += a[i];
            }
        }
        return s;
    }
};

```

9.2 mo

```

int hilbert(int x, int y) {
    int h = 0;
    for (int s = 25; s >= 0; --s) {
        int rx = (x >> s) & 1;
        int ry = (y >> s) & 1;
        int qd = (3 * rx) ^ ry;
        h = (h << 2) | qd;
        if (ry == 0) {
            x ^= -rx;
            y ^= -ry;
            swap(x, y);
        }
    }
    return h;
}

vector<int> mo(vector<int> &a, vector<array<int, 4>> &q) {
    for (auto &[h, l, r, id] : q) h = hilbert(l, r);
    sort(q.begin(), q.end());

    int curr = 0;
    auto insert = [&](int x) -> void {};
    auto remove = [&](int x) -> void {};

    vector<int> ans(q.size());
    int i = 0, j = -1;
    for (auto &[h, l, r, id] : q) {
        while (i > l) insert(a[--i]);
        while (j < r) insert(a[++j]);
        while (i < l) remove(a[i--]);
        while (j > r) remove(a[j--]);
        ans[id] = curr;
    }
}

```

```

}
return ans;
}

```

9.3 mo-3d

```

// Mo's Algorithm 3D: offline range queries with point updates
// Time: O(N^(5/3)) | Space: O(N + Q + U)
// Q elements: {l, r, t, id} | t = # of updates prior to query | id =
//       original query index
// U elements: {i, x} | i = array index to update | x = new value
// M parameter: block size multiplier to tune the N^(2/3) block size
// Note: implement 'insert', 'remove', and 'curr' state according to
//       your specific problem
vector<int> mo_3d(vector<int> &a, vector<array<int, 4>> &q,
    vector<array<int, 2>> &u, float m = 1.0) {
    int b = clamp<int>(m * pow(a.size(), 2.0 / 3.0), 1, a.size());
    auto block = [&](int i) -> int {
        return i / b;
    };

    sort(q.begin(), q.end(), [&](const array<int, 4> &x, const
        array<int, 4> &y) {
        if (block(x[0]) != block(y[0])) return block(x[0]) <
            block(y[0]);
        if (block(x[1]) != block(y[1])) return (block(x[0]) & 1) ?
            block(x[1]) < block(y[1]) : block(x[1]) > block(y[1]);
        return (block(x[1]) & 1) ? x[2] < y[2] : x[2] > y[2];
    });

    int curr = 0;
    auto insert = [&](int x) -> void {};
    auto remove = [&](int x) -> void {};

    auto update = [&](int t, int l, int r) -> void {
        int i = u[t][0];
        if (l <= i and i <= r) {
            remove(a[i]);
            swap(a[i], u[t][1]);
            insert(a[i]);
        } else {
            swap(a[i], u[t][1]);
        }
    };

    vector<int> ans(q.size());
    int h = 0, i = 0, j = -1;
    for (auto &[l, r, t, id] : q) {
        while (h < t) update(h++, i, j);
        while (h > t) update(--h, i, j);
        while (i > l) insert(a[--i]);
        while (j < r) insert(a[++j]);
        while (i < l) remove(a[i--]);
        while (j > r) remove(a[j--]);
        ans[id] = curr;
    }
    return ans;
}

```

10 string

10.1 aho-corasick

```

struct aho_corasick {
    struct node {
        int fail = 0;
        int exit = 0;
        int len = 0;
        int cnt = 0;
        array<int, 26> next = {};
    };

    vector<node> trie;

    aho_corasick(int n = 0) {
        trie.reserve(n + 5);
        trie.emplace_back();
    }

    void insert(string &s) {
        int i = 0;
        for (char c : s) {
            int k = c - 'a';
            if (trie[i].next[k] == 0) {
                trie[i].next[k] = trie.size();
                trie.emplace_back();
            }
            i = trie[i].next[k];
        }
        trie[i].len = s.length();
        trie[i].cnt++;
    }

    void compile() {
        queue<int> q;
        for (int c = 0; c < 26; ++c) {
            int i = trie[0].next[c];
            if (i != 0) q.push(i);
        }
    }
}

```



```

    }
    while (q.size()) {
        int i = q.front(); q.pop();
        int f = trie[i].fail;
        trie[i].exit = trie[f].cnt != 0 ? f : trie[f].exit;
        for (int c = 0; c < 26; ++c) {
            int &j = trie[i].next[c];
            if (j != 0) {
                trie[j].fail = trie[f].next[c];
                q.push(j);
            } else {
                j = trie[f].next[c];
            }
        }
    }
}

vector<int> match(string &t) {
    int n = t.length();
    vector<int> p;
    int i = 0;
    for (int k = 0; k < n; ++k) {
        i = trie[i].next[t[k] - 'a'];
        for (int j = i; j != 0; j = trie[j].exit) if (trie[j].cnt != 0) {
            for (int r = 1; r <= trie[j].cnt; ++r) {
                p.push_back(k - trie[j].len + 1);
            }
        }
    }
    return p;
}
};

```

10.2 deterministic-finite-automaton

```

vector<vector<int>> deterministic_finite_automaton(string &s) {
    int n = s.length();
    vector<int> lps = longest_prefix_suffix(s);
    vector<vector<int>> dfa(n + 1, vector<int>(26));
    for (int i = 0; i <= n; ++i) {
        for (int c = 0; c < 26; ++c) {
            if (i < n and (s[i] - 'a') == c) {
                dfa[i][c] = i + 1;
            } else if (i > 0) {
                dfa[i][c] = dfa[lps[i] - 1][c];
            } else {
                dfa[i][c] = 0;
            }
        }
    }
    return dfa;
}
};

```

10.3 dynamic-hashier

```

struct dynamic_hasher {
    int n;
    vector<info> tree;

    dynamic_hasher(const string &s): n(s.length()), tree(2 * n) {
        for (int i = 0; i < n; ++i) tree[n + i] = info(s[i]);
        for (int i = n - 1; i > 0; --i) tree[i] = combine(tree[i << 1], tree[(i << 1) | 1]);
    }

    void update(int i, char c) {
        tree[i += n] = info(c);
        for (i >= 1; i > 0; i >= 1) tree[i] = combine(tree[i << 1], tree[(i << 1) | 1]);
    }

    info query(int i, int j) {
        info l, r;
        for (i += n, j += n + 1; i < j; i >= 1, j >= 1) {
            if (i & 1) l = combine(l, tree[i++]);
            if (j & 1) r = combine(tree[--j], r);
        }
        return combine(l, r);
    }
};

```

10.4 hasher

```

const array<int, 2> base = {41, 53};
const array<int, 2> mod = {1000000007, 1000000009};

const int maxn = 1e6;
array<array<int, 2>, maxn + 1> pwr;

void init() {
    pwr[0] = {1, 1};
    for (int i = 1; i <= maxn; ++i) {
        for (int j = 0; j <= 1; ++j) {
            pwr[i][j] = (pwr[i - 1][j] * base[j]) % mod[j];
        }
    }
}

struct info {
    int len;

```

```

array<int, 2> ord, rev;

info(char c = 0) {
    len = c != 0;
    for (int i = 0; i <= 1; ++i) {
        ord[i] = c % mod[i];
        rev[i] = c % mod[i];
    }
}

bool operator==(const info &o) const {
    return len == o.len and ord == o.ord;
}

};

info combine(const info &l, const info &r) {
    info m;
    m.len = l.len + r.len;
    for (int i = 0; i <= 1; ++i) {
        m.ord[i] = (l.ord[i] * pwr[r.len][i] + r.ord[i]) % mod[i];
        m.rev[i] = (r.rev[i] * pwr[l.len][i] + l.rev[i]) % mod[i];
    }
    return m;
}

struct hasher {
    int n;
    vector<array<int, 2>> pref, suf;

    hasher(string &s): n(s.length()), pref(n + 1), suf(n + 2) {
        for (int i = 0; i < n; ++i) {
            for (int j = 0; j <= 1; ++j) {
                pref[i + 1][j] = (pref[i][j] * base[j] + s[i]) % mod[j];
                suf[n - i][j] = (suf[n - i + 1][j] * base[j] + s[n - i - 1]) % mod[j];
            }
        }
    }

    info query(int l, int r) const {
        assert(l <= r);
        info h;
        h.len = r - l + 1;
        for (int i = 0; i <= 1; ++i) {
            h.ord[i] = (pref[r + 1][i] - pref[l][i] * pwr[h.len][i]) % mod[i];
            h.ord[i] = (h.ord[i] + mod[i]) % mod[i];
            h.rev[i] = (suf[l + 1][i] - suf[r + 2][i] * pwr[h.len][i]) % mod[i];
            h.rev[i] = (h.rev[i] + mod[i]) % mod[i];
        }
        return h;
    }
};

```

10.5 longest-prefix-suffix

```

vector<int> longest_prefix_suffix(string &s) {
    int n = s.length();
    vector<int> lps(n);
    for (int i = 1; i < n; ++i) {
        int j = lps[i - 1];
        while (j != 0 and s[i] != s[j]) j = lps[j - 1];
        if (s[i] == s[j]) j++;
        lps[i] = j;
    }
    return lps;
}

```

10.6 manacher

```

string pre_process(string &s) {
    string t = "#";
    for (char &c : s) {
        t.push_back(c);
        t.push_back('#');
    }
    return t;
}

vector<int> manacher(string &s) {
    string t = '^' + pre_process(s) + '$';
    int n = t.length();

    vector<int> p(n);
    int c = 0, r = 0;
    for (int i = 1; i < n - 1; i++) {
        int j = 2 * c - i;
        if (i < r) {
            p[i] = min(r - i, p[j]);
        }
        while (t[i + (p[i] + 1)] == t[i - (p[i] + 1)]) {
            p[i]++;
        }
        if (i + p[i] > r) {
            c = i;
            r = i + p[i];
        }
    }
    return p;
}

```

```

}

string longest_palindrome(string &s) {
    vector<int> p = manacher(s);
    int n = p.size();

    int k = 0, l = 0;
    for (int i = 1; i < n - 1; i++) {
        if (p[i] > 1) {
            l = p[i];
            k = (i - 1) / 2;
        }
    }
    return s.substr(k, l);
}

int count_palindromes(string &s) {
    vector<int> p = manacher(s);
    int n = p.size();

    int total = 0;
    for (int i = 1; i < n - 1; ++i) {
        total += (p[i] + 1) / 2;
    }
    return total;
}

```

10.7 palindromic-tree

```

struct palindromic_tree {
    struct node {
        int length, fail;
        array<int, 26> child = {};

        node(int length): length(length), fail(0) {}
    };

    int suffix;
    vector<char> text;
    vector<node> tree;

    palindromic_tree(int n): suffix(0) {
        text.reserve(n + 5);
        text.push_back('$');
        tree.reserve(n + 5);
        tree.push_back(node(0));
        tree.push_back(node(-1));
        tree[0].fail = 1;
    }

    int follow(int id) {
        while (text[(text.size() - tree[id].length) - 2] !=
            text.back()) {
            id = tree[id].fail;
        }
        return id;
    }

    void insert(char c) {
        text.push_back(c);
        suffix = follow(suffix);

        if (not tree[suffix].child[c - 'a']) {
            int id = tree.size();
            tree.push_back(node(tree[suffix].length + 2));

            tree[id].fail = tree[follow(tree[suffix].fail)].child[c -
                'a'];
            tree[suffix].child[c - 'a'] = id;
        }
        suffix = tree[suffix].child[c - 'a'];
    }
};

```

10.8 suffix-array

```

// Suffix Array: lexicographical sorting of all string suffixes
// Time: build O(N log N) | LCP O(N) | search O(|P| log N) | Space:
//       O(N)
// Note: expects sentinel character at the end of input | supports
//       pattern matching and k-th queries
struct suffix_array {
    int n, m;
    string s;
    vector<int> suf, lcp;
    vector<int> rank, temp;

    void sort() {
        vector<int> f(m + 1);
        for (int i = 0; i < n; ++i) {
            int r = rank[suf[i]];
            f[r] += 1;
        }
        for (int i = 1; i <= m; ++i) {
            f[i] += f[i - 1];
        }
        for (int i = n - 1; i >= 0; --i) {
            int r = rank[suf[i]];
            f[r] -= 1;
            temp[f[r]] = suf[i];
        }
    }
}

```

```

        swap(suf, temp);
    }

    suffix_array(string &s): n(s.length()), m(0), s(s), suf(n),
        lcp(n), rank(n), temp(n) {
        for (int i = 0; i < n; ++i) {
            suf[i] = i;
            rank[i] = s[i];
            m = max<int>(m, s[i]);
        }
        sort();
        for (int k = 1; k < n; k <= 1) {
            for (int i = 0; i < n; ++i) {
                suf[i] = (suf[i] - k + n) % n;
            }
            sort();
            temp[suf[0]] = 1;
            for (int i = 1; i < n; ++i) {
                int prev = suf[i - 1];
                int curr = suf[i];
                if (rank[prev] == rank[curr] and rank[(prev + k) % n]
                    == rank[(curr + k) % n]) {
                    temp[curr] = temp[prev];
                } else {
                    temp[curr] = temp[prev] + 1;
                }
            }
            swap(rank, temp);
            m = rank[suf[n - 1]];
            if (m == n) break;
        }
    }

    void longest_common_prefix() {
        vector<int> inv(n);
        for (int i = 0; i < n; ++i) {
            inv[suf[i]] = i;
        }
        int l = 0;
        for (int i = 0; i < n; ++i) {
            if (inv[i] == 0) {
                l = 0;
                continue;
            }
            int j = suf[inv[i] - 1];
            while (i + l < n and j + l < n and s[i + l] == s[j + l])
                l++;
            lcp[inv[i]] = l;
            l = max<int>(0, l - 1);
        }
    }

    int compare(int i, string &t) {
        return s.compare(i, t.size(), t);
    }

    int lower_search(string &t) {
        int answer = 0;
        int low = 1, high = n - 1;
        while (low <= high) {
            int mid = (low + high) / 2;
            if (compare(suf[mid], t) < 0) {
                low = mid + 1;
            } else {
                answer = mid;
                high = mid - 1;
            }
        }
        return answer;
    }

    int upper_search(string &t) {
        int answer = n;
        int low = 1, high = n - 1;
        while (low <= high) {
            int mid = (low + high) / 2;
            if (compare(suf[mid], t) <= 0) {
                low = mid + 1;
            } else {
                answer = mid;
                high = mid - 1;
            }
        }
        return answer;
    }

    int search(string &t) {
        int lower = lower_search(t);
        if (compare(suf[lower], t) != 0) return 0;
        int upper = upper_search(t);
        return upper - lower;
    }

    pair<int, int> kth_distinct_substring(int k) {
        assert(k >= 0);
        for (int i = 1; i < n; ++i) {
            int len = (n - 1) - suf[i];
            int sub = len - lcp[i];
            if (k < sub) return {suf[i], lcp[i] + k + 1};
            k -= sub;
        }
    }
}

```

```

    return {-1, -1};
}

pair<int, int> kth_substring(int k) {
    assert(k >= 0);
    sparse_table st(lcp);
    for (int i = 1; i < n; ++i) {
        int curr_len = lcp[i] + 1;
        int max_len = (n - 1) - suf[i];
        while (curr_len <= max_len) {
            int j = i;
            int low = i + 1, high = n - 1;
            while (low <= high) {
                int mid = (low + high) / 2;
                if (st.query(i + 1, mid) >= curr_len) {
                    j = mid;
                    low = mid + 1;
                } else {
                    high = mid - 1;
                }
            }
            int freq = j - i + 1;
            int len = max_len;
            if (j > i) len = min(len, st.query(i + 1, j));
            int sub = (len - curr_len + 1) * freq;
            if (k < sub) return {suf[i], curr_len + (k / freq)};
            k -= sub;
            curr_len = len + 1;
        }
    }
    return {-1, -1};
}
};

```

10.9 suffix-automaton

```

// Suffix Automaton: directed acyclic graph representing all
// substrings of a string
// Time: extend O(1) amortized | Space: O(N * S)
// Note: root at index 1 | alphabet [a-z]
struct suffix_automaton {
    struct node {
        int len = 0, link = 0;
        int pos = -1, cnt = 0;
        array<int, 26> next = {};
    };

    int prev;
    vector<node> st;

    suffix_automaton(): prev(1), st(2) {}

    void extend(char c) {
        int idx = c - 'a';
        int curr = st.size();
        int ptr = prev;

        st.emplace_back();
        st[curr].len = st[prev].len + 1;
        st[curr].pos = st[curr].len - 1;
        st[curr].cnt = 1;
        prev = curr;

        while (ptr and (not st[ptr].next[idx])) {
            st[ptr].next[idx] = curr;
            ptr = st[ptr].link;
        }

        if (not ptr) {
            st[curr].link = 1;
            return;
        }

        int qtr = st[ptr].next[idx];
        if (st[ptr].len + 1 == st[qtr].len) {
            st[curr].link = qtr;
            return;
        }

        int clone = st.size();
        st.push_back(st[qtr]);
        st[clone].len = st[ptr].len + 1;
        st[clone].cnt = 0;

        while (ptr and st[ptr].next[idx] == qtr) {
            st[ptr].next[idx] = clone;
            ptr = st[ptr].link;
        }

        st[curr].link = st[qtr].link = clone;
    }

    bool contains(string &t) {
        int ptr = 1;
        for (char c : t) {
            int idx = c - 'a';
            ptr = st[ptr].next[idx];
            if (not ptr) return false;
        }
        return true;
    }
};

```

```
};
```

10.10 tree-suffix-array

```

// Tree Suffix Array: lexicographical sorting of all tree paths (node
// to root)
// Time: build O(N log N) | LCP O(log N) | search O(|P| log N) |
// Space: O(N log N)
// Note: 1-indexed nodes, node 0 is reserved as a sentinel root with
// s[0] = '$'
struct tree_suffix_array {
    int n, m, log;
    string s;
    vector<int> suf, lcp, temp, depth, freq;
    vector<vector<int>> up, rank;

    void dfs(int a, int p, int d, vector<vector<int>> &g) {
        up[0][a] = p;
        depth[a] = d;
        for (int k = 1; k <= log; ++k) {
            up[k][a] = up[k - 1][up[k - 1][a]];
        }
        for (int b : g[a]) if (b != p) {
            dfs(b, a, d + 1, g);
        }
    }

    void sort(int k, bool jump) {
        fill(freq.begin(), freq.begin() + m + 1, 0);
        for (int i = 0; i < n; ++i) {
            int h = suf[i];
            if (jump) h = up[k][h];
            int r = rank[k][h];
            freq[r]++;
        }
        for (int i = 1; i <= m; ++i) {
            freq[i] += freq[i - 1];
        }
        for (int i = n - 1; i >= 0; --i) {
            int h = suf[i];
            if (jump) h = up[k][h];
            int r = rank[k][h];
            freq[r]--;
            temp[freq[r]] = suf[i];
        }
        swap(suf, temp);
    }

    tree_suffix_array(const string &letters, vector<vector<int>> &g):
        n(letters.length()), m(0), log((__lg(n | 1) + 1)),
        s(letters), suf(n), lcp(n), temp(n), depth(n), freq(max<int>(256,
        n)), up(log + 1, vector<int>(n)), rank(log + 1,
        vector<int>(n)) {
        dfs(1, 0, 1, g);
        for (int i = 0; i < n; ++i) {
            suf[i] = i;
            rank[0][i] = s[i];
            m = max<int>(m, s[i]);
        }
        for (int k = 0; k < log; ++k) {
            sort(k, true);
            sort(k, false);
            rank[k + 1][suf[0]] = 0;
            int val = 0;
            for (int i = 1; i < n; ++i) {
                int prev = suf[i - 1];
                int curr = suf[i];
                int j_prev = up[k][prev];
                int j_curr = up[k][curr];
                if (rank[k][j_prev] == rank[k][j_curr] and
                    rank[k][j_prev] == rank[k][j_curr]) {
                    rank[k + 1][curr] = val;
                } else {
                    rank[k + 1][curr] = ++val;
                }
            }
            m = val;
            if (m == n - 1) {
                for (int h = k + 1; h <= log; ++h) rank[h] = rank[k +
                1];
                break;
            }
        }
    }

    int longest_common_prefix(int a, int b) {
        if (a == b) return depth[a];
        int answer = 0;
        for (int k = log; k >= 0 and a != 0 and b != 0; --k) if
            (rank[k][a] == rank[k][b]) {
                answer += 1 << k;
                a = up[k][a];
                b = up[k][b];
            }
        return answer;
    }

    void build_longest_common_prefix() {
        for (int i = 1; i < n; ++i) {
            lcp[i] = longest_common_prefix(suf[i], suf[i - 1]);
        }
    }
};

```

```

}

int compare(int a, string &t) {
    for (int i = 0; i < t.length(); ++i) {
        if (a == 0) return -1;
        if (s[a] != t[i]) return s[a] < t[i] ? -1 : +1;
        a = up[0][a];
    }
    return 0;
}

int lower_search(string &t) {
    int answer = 0;
    int low = 1, high = n - 1;
    while (low <= high) {
        int mid = (low + high) / 2;
        if (compare(suf[mid], t) < 0) {
            low = mid + 1;
        } else {
            answer = mid;
            high = mid - 1;
        }
    }
    return answer;
}

int upper_search(string &t) {
    int answer = n;
    int low = 1, high = n - 1;
    while (low <= high) {
        int mid = (low + high) / 2;
        if (compare(suf[mid], t) <= 0) {
            low = mid + 1;
        } else {
            answer = mid;
            high = mid - 1;
        }
    }
    return answer;
}

int search(string &t) {
    int lower = lower_search(t);
    if (compare(suf[lower], t) != 0) return 0;
    int upper = upper_search(t);
    return upper - lower;
}
};

```

10.11 trie

```

struct trie {
    struct node {
        int pref = 0;
        int end = 0;
        array<int, 26> child = {};
    };

    vector<node> tree;

    trie(int n = 0) {
        tree.reserve(n + 1);
        tree.emplace_back();
    }

    void insert(string &s) {
        int i = 0;
        tree[0].pref++;
        for (char c : s) {
            int k = c - 'a';
            if (tree[i].child[k] == 0) {
                tree[i].child[k] = tree.size();
                tree.emplace_back();
            }
            i = tree[i].child[k];
            tree[i].pref++;
        }
        tree[i].end++;
    }

    void remove(string &s) {
        if (not contains(s)) return;
        int i = 0;
        tree[i].pref--;
        for (char c : s) {
            int k = c - 'a';
            i = tree[i].child[k];
            tree[i].pref--;
        }
        tree[i].end--;
    }

    bool contains(string &s) {
        int i = 0;
        for (char c : s) {
            int k = c - 'a';
            i = tree[i].child[k];
            if (i == 0) return false;
        }
        return tree[i].end != 0;
    }
};

```

```

};

```

10.12 z-array

```

vector<int> z_array(string &s) {
    int n = s.length();
    vector<int> z(n);
    for (int i = 1, l = 0, r = 0; i < n; ++i) {
        z[i] = max(0, min(z[i - l], r - (i + 1)));
        while (i + z[i] < n and s[i + z[i]] == s[z[i]]) {
            l = i;
            r = i + z[i];
            z[i] += 1;
        }
    }
    return z;
}

```

11 structures

11.1 binary-indexed-tree

```

struct bit {
    int n;
    vector<int> tree;

    static int lsb(int i) {
        return i & -i;
    }

    bit(int n = 0): n(n), tree(n + 1) {}

    void update(int i, int x) {
        for (i += 1; i <= n; i += lsb(i)) tree[i] += x;
    }

    int prefix(int i) {
        int s = 0;
        for (i += 1; i > 0; i -= lsb(i)) s += tree[i];
        return s;
    }

    int query(int i, int j) {
        return prefix(j) - prefix(i - 1);
    }

    int find_kth(int k) {
        k += 1;
        assert(k > 0);
        int i = 0;
        for (int s = 1 << __lg(n); s > 0; s >>= 1) if (i + s <= n and
            tree[i + s] < k) {
            i += s;
            k -= tree[i];
        }
        return i;
    }
};

```

11.2 binary-indexed-tree-2d

```

struct bit_2d {
    int n, m;
    vector<bit> tree;

    bit_2d(int n = 0, int m = 0): n(n), m(m), tree(n + 1, bit(m)) {}

    void build(vector<vector<int>> &a) {
        for (int i = 0; i < n; ++i) {
            for (int j = 0; j < m; ++j) {
                update(i, j, a[i][j]);
            }
        }
    }

    void update(int i, int j, int x) {
        for (i += 1; i <= n; i += i & -i) tree[i].update(j, x);
    }

    int prefix(int i, int j) {
        int s = 0;
        for (i += 1; i > 0; i -= i & -i) s += tree[i].prefix(j);
        return s;
    }

    int query(int i, int j, int p, int q) {
        return prefix(p, q)
            - prefix(i - 1, q)
            - prefix(p, j - 1)
            + prefix(i - 1, j - 1);
    }
};

```

11.3 range-binary-indexed-tree

```

struct range_bit {
    bit m, s;
};

```

```
range_bit(int n): m(n + 1), s(n + 1) {}
```

```
void update(int i, int j, int x) {
    m.update(i, x);
    m.update(j + 1, -x);
    s.update(i, x * (i - 1));
    s.update(j + 1, -x * j);
}

int prefix(int i) {
    return i * m.prefix(i) - s.prefix(i);
}

int query(int i, int j) {
    return prefix(j) - prefix(i - 1);
}
};
```

11.4 sparse-binary-indexed-tree

```
// Sparse BIT: 2D Binary Indexed Tree with Static Coordinate
// Compression
// Time: Build O(Q log^2 Q) | Update/Query O(log^2 Q) | Space: O(Q log Q)
// Queries: {type, i1, j1, i2, j2} | 'U' = Update, 'Q' = Query
struct sparse_bit {
    int n;
    vector<int> is;
    vector<vector<int>> js;
    vector<bit> tree;

    void compress(vector<int> &a) {
        sort(a.begin(), a.end());
        a.erase(unique(a.begin(), a.end()), a.end());
    }

    int id(vector<int> &a, int i) {
        return lower_bound(a.begin(), a.end(), i) - a.begin();
    }

    sparse_bit(vector<tuple<char, int, int, int, int>> &q) {
        for (auto [t, i1, j1, i2, j2] : q) {
            if (t == 'U') is.push_back(i1);
            else if (t == 'Q') is.push_back(i1 - 1), is.push_back(i2);
        }
        compress(is);
        n = is.size();

        js.resize(n + 1);
        for (auto [t, i1, j1, i2, j2] : q) {
            if (t == 'U') {
                for (i1 = id(is, i1) + 1; i1 <= n; i1 += bit::lsb(i1)) js[i1].push_back(j1);
            } else if (t == 'Q') {
                for (i1 = id(is, i1 - 1) + 1; i1 > 0; i1 -= bit::lsb(i1)) js[i1].push_back(j1 - 1), js[i1].push_back(j2);
                for (i2 = id(is, i2) + 1; i2 > 0; i2 -= bit::lsb(i2)) js[i2].push_back(j1 - 1), js[i2].push_back(j2);
            }
        }

        tree.resize(n + 1);
        for (int i = 1; i <= n; ++i) {
            compress(js[i]);
            tree[i] = bit(js[i].size());
        }
    }

    void update(int i, int j, int x) {
        for (i = id(is, i) + 1; i <= n; i += bit::lsb(i))
            tree[i].update(id(js[i], j), x);
    }

    int prefix(int i, int j) {
        int s = 0;
        for (i = id(is, i) + 1; i > 0; i -= bit::lsb(i)) s += tree[i].prefix(id(js[i], j));
        return s;
    }

    int query(int i1, int j1, int i2, int j2) {
        return prefix(i2, j2) - prefix(i1 - 1, j2) - prefix(i2, j1 - 1) + prefix(i1 - 1, j1 - 1);
    }
};
```

11.5 cover-tree

```
// Cover Tree: calculates total length covered by at least one
// interval
// Time: update O(log N), query O(1) | Space: O(N)
struct cover_tree {
    int n;
    vector<int> all, cnt, len;

    void recalc(int i) {
        if (cnt[i] > 0) len[i] = all[i];
        else if (i < n) len[i] = len[i << 1] + len[i << 1 | 1];
        else len[i] = 0;
    }
};
```

```
cover_tree(vector<int> &x): n(x.size() - 1), all(2 * n), cnt(2 * n), len(2 * n) {
    for (int i = 0; i < n; ++i) all[n + i] = x[i + 1] - x[i];
    for (int i = n - 1; i > 0; --i) all[i] = all[i << 1] + all[i << 1 | 1];
}
```

```
void update(int i, int j, int x) {
    int p = i, q = j;
    for (i += n, j += n; i < j; i >= 1, j >= 1) {
        if (i & 1) cnt[i] += x, recalc(i++);
        if (j & 1) cnt[--j] += x, recalc(j);
    }
    i = p + n;
    while (i >= 1) recalc(i);
    j = q + n - 1;
    while (j >= 1) recalc(j);
}

int query() {
    return len[1];
}
};
```

11.6 k-cover-tree

```
// Iterative K-Cover Segment Tree: calculates total length covered by
// at least K intervals
// Time: update O(K log N), query O(1) | Space: O(N * K)
struct k_cover_tree {
    int n, k;
    vector<int> all, cnt;
    vector<vector<int>> len;

    void recalc(int i) {
        vector<int> base(k + 1);
        base[0] = all[i];
        if (i < n) for (int h = 1; h <= k; ++h) base[h] = len[i << 1][h] + len[i << 1 | 1][h];
        for (int h = 1; h <= k; ++h) len[i][h] = base[max<int>(0, h - cnt[i])];
    }

    k_cover_tree(int k, vector<int> &x): n(x.size() - 1), k(k), all(2 * n), cnt(2 * n), len(2 * n, vector<int>(k + 1)) {
        for (int i = 0; i < n; ++i) all[n + i] = x[i + 1] - x[i];
        for (int i = n - 1; i > 0; --i) all[i] = all[i << 1] + all[i << 1 | 1];
    }

    void update(int i, int j, int x) {
        int p = i, q = j;
        for (i += n, j += n; i < j; i >= 1, j >= 1) {
            if (i & 1) cnt[i] += x, recalc(i++);
            if (j & 1) cnt[--j] += x, recalc(j);
        }
        i = p + n;
        while (i >= 1) recalc(i);
        j = q + n - 1;
        while (j >= 1) recalc(j);
    }

    int query() {
        return len[1][k];
    }
};
```

11.7 xor-cover-tree

```
// XOR Cover Tree: calculates total length covered by an odd number
// of intervals
// Time: update O(log N), query O(1) | Space: O(N)
struct xor_cover_tree {
    int n;
    vector<int> all, cnt, len;

    void recalc(int i) {
        int child = 0;
        if (i < n) child = len[i << 1] + len[i << 1 | 1];
        if (cnt[i] & 1) len[i] = all[i] - child;
        else len[i] = child;
    }

    xor_cover_tree(vector<int> &x): n(x.size() - 1), all(2 * n), cnt(2 * n), len(2 * n) {
        for (int i = 0; i < n; ++i) all[n + i] = x[i + 1] - x[i];
        for (int i = n - 1; i > 0; --i) all[i] = all[i << 1] + all[i << 1 | 1];
    }

    void update(int i, int j) {
        int p = i, q = j;
        for (i += n, j += n; i < j; i >= 1, j >= 1) {
            if (i & 1) cnt[i] ^= 1, recalc(i++);
            if (j & 1) cnt[--j] ^= 1, recalc(j);
        }
        i = p + n;
        while (i >= 1) recalc(i);
        j = q + n - 1;
        while (j >= 1) recalc(j);
    }
};
```

```

    }

    int query() {
        return len[1];
    }
};

11.8 coprime-set

// Coprime Set: counts elements coprime to a given number using
// Inclusion-Exclusion
// Time: insert/remove/query  $O(2^K * (K + \log D))$  | Space:  $O(N * 2^K)$ 
// Note: P must be a vector of DISTINCT prime factors | K = size of
// P, D = unique divisors in map
struct coprime_set {
    map<int, int> n;

    vector<pair<int, int>> divisors(vector<int> &p) {
        int k = p.size();
        vector<pair<int, int>> a; a.reserve(1 << k);
        for (int m = 0; m < (1 << k); ++m) {
            int d = 1, mu = 1;
            for (int i = 0; i < k; ++i) if (m & (1 << i)) {
                d *= p[i], mu *= -1;
            }
            a.push_back({d, mu});
        }
        return a;
    }

    void insert(vector<int> &p) {
        vector<pair<int, int>> a = divisors(p);
        for (auto &[d, mu] : a) {
            n[d]++;
        }
    }

    void remove(vector<int> &p) {
        vector<pair<int, int>> a = divisors(p);
        for (auto &[d, mu] : a) {
            n[d]--;
        }
    }

    int query(vector<int> &p) {
        int t = 0;
        vector<pair<int, int>> a = divisors(p);
        for (auto &[d, mu] : a) {
            t += n[d] * mu;
        }
        return t;
    }
};

```

11.9 interval-set

```

// Interval Set: maintains disjoint intervals with dynamic active
// elements count
// Time: insert/remove  $O(\log N)$  amortized | contains  $O(\log N)$  | total
//  $O(1)$  | Space:  $O(N)$ 
// Note: discrete=true for [l, r] (r-1+1) | discrete=false for
// continuous [l, r) (r-1)
struct interval_set {
    int active = 0, off;
    set<pair<int, int>> s;

    interval_set(bool discrete): off(discrete) {}

    void insert(int l, int r) {
        assert(l < r + off);
        auto itr = s.lower_bound({l, -inf});
        if (itr != s.begin() and prev(itr)->second >= l - off) itr--;
        int nl = l;
        int nr = r;
        while (itr != s.end() and itr->first <= r + off) {
            nl = min(nl, itr->first);
            nr = max(nr, itr->second);
            active -= itr->second - itr->first + off;
            itr = s.erase(itr);
        }
        active += nr - nl + off;
        s.insert({nl, nr});
    }

    void remove(int l, int r) {
        assert(l < r + off);
        auto itr = s.lower_bound({l, -inf});
        if (itr != s.begin() and prev(itr)->second >= l) itr--;
        while (itr != s.end() and itr->first <= r) {
            int cl = itr->first;
            int cr = itr->second;
            active -= cr - cl + off;
            itr = s.erase(itr);
            if (cl < l) {
                s.insert({cl, l - off});
                active += l - cl;
            }
            if (cr > r) {
                s.insert({r + off, cr});
                active += cr - r;
            }
        }
    }
};

```

```

    }

    bool contains(int l, int r) {
        assert(l < r + off);
        auto itr = s.upper_bound({l, inf});
        if (itr == s.begin()) return false;
        itr--;
        return itr->first <= l and r <= itr->second;
    }

    int total() {
        return active;
    }
};

11.10 range-set

// Range Set: bitset optimized for range updates and queries
// Time:  $O(N/64)$  per operation | Space:  $O(N/64)$ 
// Note: uint = uint64_t (unsigned long long)
struct range_set {
    int n;
    vector<uint> data;

    range_set(int n): n(n), data((n + 63) / 64) {}

    uint from(int i) {
        return ~0ULL << (i & 63);
    }

    uint until(int i) {
        return ~0ULL >> (63 - (i & 63));
    }

    void apply(int k, bool on, uint mask) {
        if (on) data[k] |= mask;
        else data[k] &= ~mask;
    }

    void update(int l, int r, bool on) {
        int i = l >> 6, j = r >> 6;
        if (i == j) {
            apply(i, on, from(l) & until(r));
        } else {
            apply(i, on, from(l));
            for (int k = i + 1; k < j; ++k) apply(k, on, ~0ULL);
            apply(j, on, until(r));
        }
    }

    int count(int l, int r) {
        int c = 0;
        int i = l >> 6, j = r >> 6;
        if (i == j) {
            c = __builtin_popcountll(data[i] & (from(l) & until(r)));
        } else {
            c += __builtin_popcountll(data[i] & from(l));
            for (int k = i + 1; k < j; ++k) c +=
                __builtin_popcountll(data[k]);
            c += __builtin_popcountll(data[j] & until(r));
        }
        return c;
    }

    bool empty() {
        for (uint mask : data) if (mask) return false;
        return true;
    }

    int prev(int i) {
        for (int k = i >> 6; k >= 0; --k, i = 63) {
            uint mask = data[k] & until(i);
            if (mask) return (k << 6) + 63 - __builtin_clzll(mask);
        }
        return n;
    }

    int next(int i) {
        for (int k = i >> 6; k < data.size(); ++k, i = 0) {
            uint mask = data[k] & from(i);
            if (mask) return (k << 6) + __builtin_ctzll(mask);
        }
        return n;
    }

    int segments() {
        int s = 0;
        for (int k = 0; k < data.size(); ++k) {
            s += __builtin_popcountll((data[k] >> 1) & ~data[k]);
            if ((data[k] & 1) and (k == 0 or (data[k - 1] >> 63) ==
                0)) s += 1;
        }
        return s;
    }
};

11.11 venice-set

// Venice Set: multiset with global value updates
// Time:  $O(\log N)$  | Space:  $O(N)$ 

```

```

struct venice_set {
    int k = 0;
    multiset<int> s;

    void insert(int x) {
        s.insert(x - k);
    }

    void remove(int x) {
        if (auto i = s.find(x - k); i != s.end()) s.erase(i);
    }

    void update(int x) {
        k += x;
    }

    int min() {
        return *s.begin() + k;
    }
};

```

11.12 disjoint-set-union

```

struct dsu {
    vector<int> p, s;

    dsu(int n): p(n), s(n, 1) {
        iota(p.begin(), p.end(), 0);
    }

    int find(int a) {
        if (p[a] == a) return a;
        return p[a] = find(p[a]);
    }

    void unite(int a, int b) {
        a = find(a);
        b = find(b);
        if (a == b) return;
        if (s[a] < s[b]) swap(a, b);
        p[b] = a;
        s[a] += s[b];
    }
};

```

11.13 disjoint-set-union-2d

```

struct dsu_2d {
    int n, m;
    dsu d;

    dsu_2d(int n, int m): n(n), m(m), d(n * m) {}

    int id(int r, int c) {
        return r * m + c;
    }

    int find(int r, int c) {
        return d.find(id(r, c));
    }

    void unite(int r1, int c1, int r2, int c2) {
        d.unite(id(r1, c1), id(r2, c2));
    }
};

```

11.14 dynamic-connectivity

```

struct dynamic_connectivity {
private:
    int n, q;
    vector<vector<pair<int, int>>> tree;
    rollback_dsu dsu;

    void insert(pair<int, int> e, int ql, int qr, int p, int l, int r) {
        if (qr < l or ql > r) return;
        if (ql <= l and qr >= r) return void(tree[p].push_back(e));
        int m = (l + r) / 2;
        insert(e, ql, qr, 2 * p, l, m);
        insert(e, ql, qr, 2 * p + 1, m + 1, r);
    }

    void compute(vector<int> &ans, int p, int l, int r) {
        for (auto [a, b] : tree[p]) dsu.unite(a, b);
        if (l == r) {
            ans[l] = dsu.c;
        } else {
            int m = (l + r) / 2;
            compute(ans, 2 * p, l, m);
            compute(ans, 2 * p + 1, m + 1, r);
        }
        for (auto [a, b] : tree[p]) dsu.rollback();
    }

public:
    dynamic_connectivity(int n, int q): n(n), q(q), tree(4 * q), dsu(n) {}

    void insert(pair<int, int> e, int ql, int qr) {
        insert(e, ql, qr, 1, 0, q - 1);
    }
};

```

```

}

vector<int> compute() {
    vector<int> ans(q);
    compute(ans, 1, 0, q - 1);
    return ans;
}

};

```

11.15 rollback-disjoint-set-union

```

struct rollback_dsu {
    int c;
    vector<int> p, s;
    vector<pair<int, int>> h;

    rollback_dsu(int n): c(n), p(n), s(n, 1) {
        iota(p.begin(), p.end(), 0);
    }

    int find(int a) {
        while (p[a] != a) a = p[a];
        return a;
    }

    void unite(int a, int b) {
        a = find(a);
        b = find(b);
        if (a == b) {
            h.push_back({-1, -1});
        } else {
            if (s[a] < s[b]) swap(a, b);
            h.push_back({a, b});
            c--;
            p[b] = a;
            s[a] += s[b];
        }
    }

    void rollback() {
        assert(h.size());
        auto [a, b] = h.back(); h.pop_back();
        if (a != -1) {
            c++;
            p[b] = b;
            s[a] -= s[b];
        }
    }

    int state() {
        return h.size();
    }
};

```

11.16 li-chao-tree

```

// Li Chao Tree: dynamic line & segment container for maximum convex hull trick
// Time: line insert/query O(log U) | segment insert O(log^2 U) | Space: O(N log U)
// Note: computes max(ax + b) | for min: insert (-a, -b) and take -query(x)
struct li_chao_tree {
private:
    struct line {
        int a, b;

        int operator()(int x) const {
            return a * x + b;
        }
    };

    struct node {
        line f;

        int l, r;
    };

    int min_x, max_x;
    vector<node> tree;

    int create_node() {
        tree.push_back({{0, -inf}, 0, 0});
        return tree.size() - 1;
    }

    int insert_line(line f, int p, int l, int r) {
        if (p == 0) p = create_node();
        int m = l + (r - l) / 2;
        bool is_l = f(l) > tree[p].f(l);
        bool is_m = f(m) > tree[p].f(m);
        if (is_m) swap(f, tree[p].f);
        if (l == r) return p;
        if (is_l != is_m) tree[p].l = insert_line(f, tree[p].l, l, m);
        else tree[p].r = insert_line(f, tree[p].r, m + 1, r);
        return p;
    }

    int insert_segment(line f, int ql, int qr, int p, int l, int r) {
        if (p == 0) p = create_node();
        if (ql <= l and r <= qr) return insert_line(f, p, l, r);
    }
};

```

```

    int m = l + (r - l) / 2;
    if (ql <= m) tree[p].l = insert_segment(f, ql, qr, tree[p].l,
        l, m);
    if (qr > m) tree[p].r = insert_segment(f, ql, qr, tree[p].r,
        m + 1, r);
    return p;
}

int query(int x, int p, int l, int r) {
    if (p == 0) return -inf;
    int y = tree[p].f(x);
    if (l == r) return y;
    int m = l + (r - l) / 2;
    if (x <= m) y = max(y, query(x, tree[p].l, l, m));
    else y = max(y, query(x, tree[p].r, m + 1, r));
    return y;
}

public:
    li_chao_tree(int min_x, int max_x): min_x(min_x), max_x(max_x) {
        tree.push_back({{0, -inf}, 0, 0});
        create_node();
    }

    void insert_line(int a, int b) {
        insert_line({a, b}, 1, min_x, max_x);
    }

    void insert_segment(int a, int b, int l, int r) {
        insert_segment({a, b}, l, r, 1, min_x, max_x);
    }

    int query(int x) {
        return query(x, 1, min_x, max_x);
    }
};

```

11.17 line-container

```

// Line Container: dynamic convex hull trick for maximum
// Time: insert/query O(log N) | Space: O(N)
// Note: computes max(ax + b) | change '<' to '>' in line operator
// for min
struct line {
    mutable int a, b, i;

    bool operator<(const line &o) const {
        return tie(a, b) < tie(o.a, o.b);
    }

    bool operator<(const int x) const {
        return i < x;
    }
};

struct line_container : multiset<line, less<>> {
    const int inf = 1e18;

    int div(int x, int y) {
        return (x / y) - (((x ^ y) < 0) and x % y);
    }

    bool isect(iterator s, iterator t) {
        if (t == end()) return s->i = inf, false;
        if (s->a == t->a) s->i = -inf;
        else s->i = div(t->b - s->b, s->a - t->a);
        return s->i >= t->i;
    }

    void insert(int a, int b) {
        auto z = multiset::insert({a, b, 0}), y = z++, x = y;
        while (isect(y, z)) z = erase(z);
        if (x != begin() and isect(--x, y)) isect(x, y = erase(y));
        while ((y = x) != begin() and (--x)->i >= y->i) isect(x,
            erase(y));
    }

    int max(int x) {
        auto l = lower_bound(x);
        return l->a * x + l->b;
    }
};

```

11.18 dynamic-merge-sort-tree

```

// Merge Sort Tree: range queries with point updates
// Time: build O(N log^2 N) | Update/Query: O(log^2 N) | Space: O(N
    log N)
// Note: uses PBDs ordered_set | pair<int, int> handles duplicate
    values
struct merge_sort_tree {
    int n;
    vector<int> a;
    vector<ordered_set<pair<int, int>>> tree;

    merge_sort_tree(vector<int> &a): n(a.size()), a(a), tree(2 * n) {
        for (int i = 0; i < n; ++i) {
            for (int j = i + n; j > 0; j >= 1) {
                tree[j].insert({a[i], i});
            }
        }
    }
};

```

```

}

void update(int i, int x) {
    for (int j = i + n; j > 0; j >= 1) {
        tree[j].erase({a[i], i});
        tree[j].insert({x, i});
    }
    a[i] = x;
}

int less_or_equal(int i, int j, int x) {
    int c = 0;
    for (i += n, j += n + 1; i < j; i >= 1, j >= 1) {
        if (i & 1) c += tree[i++].order_of_key({x + 1, 0});
        if (j & 1) c += tree[--j].order_of_key({x + 1, 0});
    }
    return c;
}
};

```

11.19 merge-sort-tree

```

struct merge_sort_tree {
    int n;
    vector<vector<int>> tree;

    merge_sort_tree(vector<int> &a): n(a.size()), tree(2 * n) {
        for (int i = 0; i < n; ++i) tree[i + n] = {a[i]};
        for (int i = n - 1; i > 0; --i) merge(tree[i < 1].begin(),
            tree[i < 1].end(), x) - tree[i].begin(), i++,
            tree[i < 1].end(), tree[(i < 1) | 1].begin(), tree[(i
                < 1) | 1].end(), back_inserter(tree[i]));
    }

    int less_or_equal(int i, int j, int x) {
        int c = 0;
        for (i += n, j += n + 1; i < j; i >= 1, j >= 1) {
            if (i & 1) c += upper_bound(tree[i].begin(),
                tree[i].end(), x) - tree[i].begin(), i++;
            if (j & 1) --j, c += upper_bound(tree[j].begin(),
                tree[j].end(), x) - tree[j].begin();
        }
        return c;
    }
};

```

11.20 hash-table

```

#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;

struct custom_hash {
    size_t operator()(size_t x) const {
        static const size_t time =
            chrono::steady_clock::now().time_since_epoch().count();
        return (x + 0x9e3779b9) ^ time;
    }
};

template<typename T>
using hash_set = gp_hash_table<T, null_type, custom_hash>;

template<typename K, typename V>
using hash_map = gp_hash_table<K, V, custom_hash>;

```

11.21 ordered-set

```

#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;

template <typename T>
using ordered_set = tree<T, null_type, less<T>, rb_tree_tag,
    tree_order_statistics_node_update>;

```

11.22 lazy-segment-tree

```

struct lazy_segment_tree {
private:
    int n;
    vector<int> tree;
    vector<int> lazy;

    void build(vector<int> &a, int p, int l, int r) {
        if (l == r) {
            tree[p] = a[l];
        } else {
            int m = (l + r) / 2;
            build(a, 2 * p, l, m);
            build(a, (2 * p) + 1, m + 1, r);
            tree[p] = tree[2 * p] + tree[(2 * p) + 1];
        }
    }

    void apply(int p, int x, int len) {
        tree[p] += x * len;
        lazy[p] += x;
    }

    void push(int p, int l, int r) {
        if (lazy[p] != 0) {

```



```

        int m = (l + r) / 2;
        apply(2 * p, lazy[p], m - 1 + 1);
        apply((2 * p) + 1, lazy[p], r - m);
        lazy[p] = 0;
    }
}

void update(int ql, int qr, int x, int p, int l, int r) {
    if (ql <= l and r <= qr) {
        apply(p, x, r - l + 1);
    } else {
        push(p, l, r);
        int m = (l + r) / 2;
        if (ql <= m) update(ql, qr, x, 2 * p, l, m);
        if (qr > m) update(ql, qr, x, (2 * p) + 1, m + 1, r);
        tree[p] = tree[2 * p] + tree[(2 * p) + 1];
    }
}

int query(int ql, int qr, int p, int l, int r) {
    if (ql <= l and r <= qr) {
        return tree[p];
    } else {
        push(p, l, r);
        int m = (l + r) / 2;
        int answer = 0;
        if (ql <= m) answer += query(ql, qr, 2 * p, l, m);
        if (qr > m) answer += query(ql, qr, (2 * p) + 1, m + 1, r);
        return answer;
    }
}

public:
    lazy_segment_tree(int n): n(n), tree(4 * n), lazy(4 * n) {}

    lazy_segment_tree(vector<int> &a): n(a.size()), tree(4 * n),
        lazy(4 * n) {
        build(a, 1, 0, n - 1);
    }

    void update(int l, int r, int x) {
        update(l, r, x, 1, 0, n - 1);
    }

    int query(int l, int r) {
        return query(l, r, 1, 0, n - 1);
    }
};

```

11.23 lazy-sparse-segment-tree

```

struct lazy_sparse_segment_tree {
private:
    struct node {
        int val = 0, lazy = 0;
        int l = 0, r = 0;
    };

    int root = 0, min_l, max_r;
    vector<node> tree;

    void push(int p, int l, int r) {
        if (not tree[p].lazy) return;
        int m = l + (r - l) / 2;
        if (not tree[p].l) {
            tree[p].l = tree.size();
            tree.emplace_back();
        }
        if (not tree[p].r) {
            tree[p].r = tree.size();
            tree.emplace_back();
        }
        tree[tree[p].l].lazy += tree[p].lazy;
        tree[tree[p].r].lazy += tree[p].lazy;
        tree[tree[p].l].val += tree[p].lazy * (m - l + 1);
        tree[tree[p].r].val += tree[p].lazy * (r - m);
        tree[p].lazy = 0;
    }

    int update(int ql, int qr, int x, int p, int l, int r) {
        if (not p) {
            p = tree.size();
            tree.emplace_back();
        }
        if (ql <= l and r <= qr) {
            tree[p].lazy += x;
            tree[p].val += x * (r - l + 1);
            return p;
        }
        push(p, l, r);
        int m = l + (r - l) / 2;
        if (ql <= m) {
            int nl = update(ql, qr, x, tree[p].l, l, m);
            tree[p].l = nl;
        }
        if (qr > m) {
            int nr = update(ql, qr, x, tree[p].r, m + 1, r);
            tree[p].r = nr;
        }
        tree[p].val = tree[tree[p].l].val + tree[tree[p].r].val;
    }
};

```

```

        return p;
    }

    int query(int ql, int qr, int p, int l, int r) {
        if (not p or (qr < l or r < ql)) {
            return 0;
        }
        if (ql <= l and r <= qr) {
            return tree[p].val;
        }
        push(p, l, r);
        int m = l + (r - l) / 2;
        return query(ql, qr, tree[p].l, l, m) + query(ql, qr,
            tree[p].r, m + 1, r);
    }

public:
    lazy_sparse_segment_tree(int l, int r, int q = 0): min_l(l),
        max_r(r) {
        tree.reserve(1 + 35 * q);
        tree.emplace_back();
    }

    void update(int ql, int qr, int x) {
        root = update(ql, qr, x, root, min_l, max_r);
    }

    int query(int ql, int qr) {
        return query(ql, qr, root, min_l, max_r);
    }
};

```

11.24 persistent-segment-tree

```

struct persistent_segment_tree {
private:
    struct node {
        int x = 0;
        int l = 0, r = 0;
    };

    int n;
    vector<node> tree;

    int clone(int p) {
        tree.push_back(tree[p]);
        return tree.size() - 1;
    }

    int build(vector<int> &a, int l, int r) {
        int p = tree.size();
        tree.emplace_back();
        if (l == r) {
            tree[p].x = a[l];
            return p;
        }
        int m = (l + r) / 2;
        tree[p].l = build(a, l, m);
        tree[p].r = build(a, m + 1, r);
        tree[p].x = tree[tree[p].l].x + tree[tree[p].r].x;
        return p;
    }

    int update(int i, int x, int p, int l, int r) {
        int c = clone(p);
        if (l == r) {
            tree[c].x = x;
            return c;
        }
        int m = (l + r) / 2;
        if (i <= m) {
            int nl = update(i, x, tree[c].l, l, m);
            tree[c].l = nl;
        } else {
            int nr = update(i, x, tree[c].r, m + 1, r);
            tree[c].r = nr;
        }
        tree[c].x = tree[tree[c].l].x + tree[tree[c].r].x;
        return c;
    }

    int query(int ql, int qr, int p, int l, int r) {
        if (qr < l or r < ql) return 0;
        if (ql <= l and r <= qr) return tree[p].x;
        int m = (l + r) / 2;
        return query(ql, qr, tree[p].l, l, m) + query(ql, qr,
            tree[p].r, m + 1, r);
    }

    int copy_range(int ql, int qr, int s, int t, int l, int r) {
        if (qr < l or r < ql) return t;
        if (ql <= l and r <= qr) return s;
        int c = clone(t);
        int m = (l + r) / 2;
        int nl = copy_range(ql, qr, tree[s].l, tree[t].l, l, m);
        tree[c].l = nl;
        int nr = copy_range(ql, qr, tree[s].r, tree[t].r, m + 1, r);
        tree[c].r = nr;
        tree[c].x = tree[tree[c].l].x + tree[tree[c].r].x;
        return c;
    }
};

```

```

public:
    vector<int> roots;

    persistent_segment_tree(vector<int> &a, int q = 0): n(a.size()) {
        tree.reserve(4 * n + 20 * q);
        roots.push_back(build(a, 0, n - 1));
    }

    void update(int i, int x, int v = -1) {
        if (v == -1) v = roots.size() - 1;
        roots.push_back(update(i, x, roots[v], 0, n - 1));
    }

    int query(int ql, int qr, int v = -1) {
        if (v == -1) v = roots.size() - 1;
        return query(ql, qr, roots[v], 0, n - 1);
    }

    void copy_range(int ql, int qr, int s, int t = -1) {
        if (t == -1) t = roots.size() - 1;
        roots.push_back(copy_range(ql, qr, roots[s], roots[t], 0, n - 1));
    }
};

```

11.25 recursive-segment-tree

```

struct segment_tree {
private:
    int n;
    vector<int> tree;

    void build(vector<int> &a, int p, int l, int r) {
        if (l == r) {
            tree[p] = a[l];
        } else {
            int m = (l + r) / 2;
            build(a, 2 * p, l, m);
            build(a, (2 * p) + 1, m + 1, r);
            tree[p] = tree[2 * p] + tree[(2 * p) + 1];
        }
    }

    void update(int i, int x, int p, int l, int r) {
        if (l == r) {
            tree[p] = x;
        } else {
            int m = (l + r) / 2;
            if (i <= m) update(i, x, 2 * p, l, m);
            if (i > m) update(i, x, (2 * p) + 1, m + 1, r);
            tree[p] = tree[2 * p] + tree[(2 * p) + 1];
        }
    }

    int query(int ql, int qr, int p, int l, int r) {
        if (ql <= l and r <= qr) {
            return tree[p];
        } else {
            int m = (l + r) / 2;
            int answer = 0;
            if (ql <= m) answer += query(ql, qr, 2 * p, l, m);
            if (qr > m) answer += query(ql, qr, (2 * p) + 1, m + 1, r);
            return answer;
        }
    }
public:
    segment_tree(int n): n(n), tree(4 * n) {}

    segment_tree(vector<int> &a): n(a.size()), tree(4 * n) {
        build(a, 1, 0, n - 1);
    }

    void update(int i, int x) {
        update(i, x, 1, 0, n - 1);
    }

    int query(int l, int r) {
        return query(l, r, 1, 0, n - 1);
    }
};

```

11.26 reversible-segment-tree

```

struct reversible_segment_tree {
    int n;
    bool v;
    vector<int> tree;

    int merge(int x, int y) {
        return x + y;
    }

    int combine(int x, int y) {
        if (v) return merge(y, x);
        return merge(x, y);
    }

    reversible_segment_tree(int s, bool v): n(_bit_ceil(s)), v(v),

```

```

    tree(2 * n) {}

    reversible_segment_tree(vector<int> &a, bool v):
        reversible_segment_tree(a.size(), v) {
        for (int i = 0; i < a.size(); ++i) tree[n + i] = a[i];
        for (int i = n - 1; i > 0; --i) tree[i] = combine(tree[i << 1], tree[(i << 1) | 1]);
    }

    void update(int i, int x) {
        tree[i += n] = x;
        while (i >= 1) tree[i] = combine(tree[i << 1], tree[(i << 1) | 1]);
    }

    int query(int i, int j) {
        int l = 0, r = 0;
        for (i += n, j += n + 1; i < j; i >= 1, j >= 1) {
            if (i & 1) l = combine(l, tree[i++]);
            if (j & 1) r = combine(tree[--j], r);
        }
        return combine(l, r);
    }
};

```

11.27 segment-tree

```

struct segment_tree {
    int n;
    vector<int> tree;

    int merge(int x, int y) {
        return x + y;
    }

    segment_tree(int s): n(_bit_ceil(s)), tree(2 * n) {}

    segment_tree(vector<int> &a): segment_tree(a.size()) {
        for (int i = 0; i < a.size(); ++i) tree[n + i] = a[i];
        for (int i = n - 1; i > 0; --i) tree[i] = merge(tree[i << 1], tree[(i << 1) | 1]);
    }

    void update(int i, int x) {
        tree[i += n] = x;
        while (i >= 1) tree[i] = merge(tree[i << 1], tree[(i << 1) | 1]);
    }

    int query(int i, int j) {
        int l = 0, r = 0;
        for (i += n, j += n + 1; i < j; i >= 1, j >= 1) {
            if (i & 1) l = merge(l, tree[i++]);
            if (j & 1) r = merge(tree[--j], r);
        }
        return merge(l, r);
    }
};

```

11.28 segment-tree-2d

```

struct segment_tree_2d {
    int n, m;
    vector<segment_tree> tree;

    segment_tree_2d(int n, int m): n(n), m(m), tree(2 * n, segment_tree(m)) {}

    segment_tree_2d(vector<vector<int>> &a):
        segment_tree_2d(a.size(), a[0].size()) {
        for (int i = 0; i < n; ++i) tree[n + i] = segment_tree(a[i]);
        for (int i = n - 1; i > 0; --i) {
            for (int j = 1; j < 2 * m; ++j) {
                tree[i].tree[j] = segment_tree::merge(tree[i << 1].tree[j], tree[(i << 1) | 1].tree[j]);
            }
        }
    }

    void update(int i, int j, int x) {
        tree[i += n].update(j, x);
        while (i >= 1) tree[i].update(j, segment_tree::merge(tree[i << 1].tree[j + m], tree[(i << 1) | 1].tree[j + m]));
    }

    int query(int i1, int j1, int i2, int j2) {
        int l = 0, r = 0;
        for (i1 += n, i2 += n + 1; i1 < i2; i1 >= 1, i2 >= 1) {
            if (i1 & 1) l = segment_tree::merge(l, tree[i1++].query(j1, j2));
            if (i2 & 1) r = segment_tree::merge(tree[--i2].query(j1, j2), r);
        }
        return segment_tree::merge(l, r);
    }
};

```

11.29 sparse-persistent-segment-tree

```

struct sparse_persistent_segment_tree {
private:

```

```

struct node {
    int x = 0;
    int l = 0, r = 0;
};

int min_l, max_r;
vector<node> tree;

int clone(int p) {
    tree.push_back(tree[p]);
    return tree.size() - 1;
}

int update(int i, int x, int p, int l, int r) {
    int c = clone(p);
    if (l == r) {
        tree[c].x = x;
        return c;
    }
    int m = l + (r - l) / 2;
    if (i <= m) {
        int nl = update(i, x, tree[c].l, l, m);
        tree[c].l = nl;
    } else {
        int nr = update(i, x, tree[c].r, m + 1, r);
        tree[c].r = nr;
    }
    tree[c].x = tree[tree[c].l].x + tree[tree[c].r].x;
    return c;
}

int query(int ql, int qr, int p, int l, int r) {
    if (not p or (qr < l or r < ql)) return 0;
    if (ql <= l and r <= qr) return tree[p].x;
    int m = l + (r - l) / 2;
    return query(ql, qr, tree[p].l, l, m) + query(ql, qr,
        tree[p].r, m + 1, r);
}

public:
vector<int> roots;

sparse_persistent_segment_tree(int l, int r, int q = 0):
    min_l(l), max_r(r) {
    tree.reserve(1 + 35 * q);
    tree.emplace_back();
    roots.push_back(0);
}

void update(int i, int x, int v = -1) {
    if (v == -1) v = roots.size() - 1;
    roots.push_back(update(i, x, roots[v], min_l, max_r));
}

int query(int ql, int qr, int v = -1) {
    if (v == -1) v = roots.size() - 1;
    return query(ql, qr, roots[v], min_l, max_r);
}
};

```

11.30 sparse-segment-tree

```

struct sparse_segment_tree {
private:
    struct node {
        int x = 0;
        int l = 0, r = 0;
    };

    int root = 0, min_l, max_r;
    vector<node> tree;

    int update(int i, int x, int p, int l, int r) {
        if (not p) {
            p = tree.size();
            tree.emplace_back();
        }
        if (l == r) {
            tree[p].x = x;
            return p;
        }
        int m = l + (r - l) / 2;
        if (i <= m) {
            int nl = update(i, x, tree[p].l, l, m);
            tree[p].l = nl;
        } else {
            int nr = update(i, x, tree[p].r, m + 1, r);
            tree[p].r = nr;
        }

        tree[p].x = tree[tree[p].l].x + tree[tree[p].r].x;
        return p;
    }

    int query(int ql, int qr, int p, int l, int r) {
        if (not p or (qr < l or r < ql)) {
            return 0;
        }
        if (ql <= l and r <= qr) {
            return tree[p].x;
        }
    }
};

```

```

    int m = l + (r - l) / 2;
    return query(ql, qr, tree[p].l, l, m) + query(ql, qr,
        tree[p].r, m + 1, r);
}

public:
    sparse_segment_tree(int l, int r, int q = 0): min_l(l), max_r(r) {
        tree.reserve(1 + 35 * q);
        tree.emplace_back();
    }

    void update(int i, int x) {
        root = update(i, x, root, min_l, max_r);
    }

    int query(int ql, int qr) {
        return query(ql, qr, root, min_l, max_r);
    }
};

```

11.31 sparse-table

```

struct sparse_table {
    int n, l;
    vector<vector<int>> table;

    static int merge(int x, int y) {
        return gcd(x, y);
    }

    sparse_table(vector<int> &a): n(a.size()), l(_lg(n)), table(l +
        1, vector<int>(n)) {
        for (int i = 0; i < n; ++i) table[0][i] = a[i];
        for (int k = 1; k <= l; ++k) {
            for (int i = 0; i + (1 << k) <= n; ++i) {
                table[k][i] = merge(table[k - 1][i], table[k - 1][i +
                    (1 << (k - 1))]);
            }
        }

        int query(int i, int j) {
            int k = _lg(j - i + 1);
            return merge(table[k][i], table[k][j - (1 << k) + 1]);
        }
    };
};

```

11.32 sparse-table-2d

```

struct sparse_table_2d {
    int n, m, l;
    vector<vector<sparse_table>> table;

    sparse_table_2d(vector<vector<int>> &a): n(a.size()),
        m(a[0].size()), l(_lg(n)), table(l + 1) {
        for (int i = 0; i < n; ++i) {
            table[0].push_back(sparse_table(a[i]));
        }
        for (int k = 1; k <= l; ++k) {
            for (int i = 0; i + (1 << k) <= n; ++i) {
                vector<int> h(m);
                for (int j = 0; j < m; ++j) {
                    int x = table[k - 1][i].table[0][j];
                    int y = table[k - 1][i + (1 << (k -
                        1))].table[0][j];
                    h[j] = sparse_table::merge(x, y);
                }
                table[k].push_back(sparse_table(h));
            }
        }

        int query(int i1, int j1, int i2, int j2) {
            int k = _lg(i2 - i1 + 1);
            int x = table[k][i1].query(j1, j2);
            int y = table[k][i2 - (1 << k) + 1].query(j1, j2);
            return sparse_table::merge(x, y);
        }
    };
};

```

11.33 implicit-treap

```

// Implicit Treap: dynamic array with range updates and queries
// Time: operations O(log N) average | Space: O(N)
// Note: supports range modifications | uses lazy propagation and
//       implicit indices
struct treap {
    struct node {
        int x, p, l = 0, r = 0;
        int sum, size = 1, lazy = 0, rev = 0;

        node(int x, int p = rng()): x(x), p(p), sum(x) {}
    };

    int root = 0;
    vector<node> t;

    treap(int n = 0) {
        t.reserve(n + 1);
        t.push_back(node(0));
        t[0].size = 0;
    }
};

```

```

}

void apply(int i, int x) {
    if (i == 0) return;
    t[i].x += x;
    t[i].sum += x * t[i].size;
    t[i].lazy += x;
}

void flip(int i) {
    if (i == 0) return;
    t[i].rev ^= 1;
    swap(t[i].l, t[i].r);
}

void push(int i) {
    if (i == 0) return;
    if (t[i].rev != 0) {
        flip(t[i].l);
        flip(t[i].r);
        t[i].rev = 0;
    }
    if (t[i].lazy != 0) {
        apply(t[i].l, t[i].lazy);
        apply(t[i].r, t[i].lazy);
        t[i].lazy = 0;
    }
}

void pull(int i) {
    if (i == 0) return;
    t[i].size = t[t[i].l].size + 1 + t[t[i].r].size;
    t[i].sum = t[t[i].l].sum + t[i].x + t[t[i].r].sum;
}

pair<int, int> split(int i, int k) {
    if (i == 0) return {0, 0};
    push(i);
    int s = t[t[i].l].size;
    if (s >= k) {
        auto [l, r] = split(t[i].l, k);
        t[i].l = r;
        pull(i);
        return {l, i};
    } else {
        auto [l, r] = split(t[i].r, k - s - 1);
        t[i].r = l;
        pull(i);
        return {i, r};
    }
}

int merge(int l, int r) {
    if (l == 0 or r == 0) return l | r;
    if (t[l].p > t[r].p) {
        push(l);
        t[l].r = merge(t[l].r, r);
        pull(l);
        return l;
    } else {
        push(r);
        t[r].l = merge(l, t[r].l);
        pull(r);
        return r;
    }
}

void insert(int k, int x) {
    auto [l, r] = split(root, k);
    int i = t.size();
    t.push_back(node(x));
    root = merge(merge(l, i), r);
}

void erase(int k) {
    auto [l, m] = split(root, k);
    auto [_, r] = split(m, 1);
    root = merge(l, r);
}

void move(int i, int j, int k) {
    auto [h, q] = split(root, j + 1);
    auto [p, m] = split(h, i);
    auto [l, r] = split(merge(p, q), k);
    root = merge(merge(l, m), r);
}

void reverse(int i, int j) {
    auto [h, r] = split(root, j + 1);
    auto [l, m] = split(h, i);
    flip(m);
    root = merge(merge(l, m), r);
}

void update(int i, int j, int x) {
    auto [h, r] = split(root, j + 1);
    auto [l, m] = split(h, i);
    apply(m, x);
    root = merge(merge(l, m), r);
}

```

```

int query(int i, int j) {
    auto [h, r] = split(root, j + 1);
    auto [l, m] = split(h, i);
    int q = t[m].sum;
    root = merge(merge(l, m), r);
    return q;
}
};

```

11.34 treap

```

// Treap: randomized binary search tree (split/merge)
// Time: operations O(log N) average | Space: O(N)
// Note: combines bst and heap properties | requires mt19937 for
//       priorities
struct treap {
    struct node {
        int x, p, l = 0, r = 0, s = 1;

        node(int x, int p = rng()): x(x), p(p) {}
    };

    int root = 0;
    vector<node> t;

    treap(int n = 0) {
        t.reserve(n + 1);
        t.push_back(node(0));
        t[0].s = 0;
    }

    void pull(int i) {
        if (i == 0) return;
        t[i].s = 1 + t[t[i].l].s + t[t[i].r].s;
    }

    pair<int, int> split(int i, int x) {
        if (i == 0) return {0, 0};
        if (t[i].x <= x) {
            auto [l, r] = split(t[i].r, x);
            t[i].r = l;
            pull(i);
            return {i, r};
        } else {
            auto [l, r] = split(t[i].l, x);
            t[i].l = r;
            pull(i);
            return {l, i};
        }
    }

    int merge(int l, int r) {
        if (l == 0 or r == 0) return l | r;
        if (t[l].p > t[r].p) {
            t[l].r = merge(t[l].r, r);
            pull(l);
            return l;
        } else {
            t[r].l = merge(l, t[r].l);
            pull(r);
            return r;
        }
    }

    void insert(int x) {
        auto [l, r] = split(root, x);
        int i = t.size();
        t.push_back(node(x));
        root = merge(merge(l, i), r);
    }

    void erase(int x) {
        auto [l, h] = split(root, x - 1);
        auto [m, r] = split(h, x);
        m = merge(t[m].l, t[m].r);
        root = merge(merge(l, m), r);
    }

    int index(int x) {
        int i = root, k = 0;
        while (i != 0) {
            if (t[i].x < x) k += 1 + t[t[i].l].s, i = t[i].r;
            else i = t[i].l;
        }
        return k;
    }

    int kth(int k) {
        int i = root;
        while (i != 0) {
            int s = t[t[i].l].s;
            if (k < s) i = t[i].l;
            else if (k > s) k -= 1 + s, i = t[i].r;
            else return t[i].x;
        }
        return -1;
    }
};

```

11.35 wavelet-matrix

```

// Wavelet Matrix: range quantile, range frequency, k-th smallest
// Time: build O(N log V) | Queries O(log V) | Space: O(N log V)
// Note: coordinate compression is REQUIRED before building.
struct wavelet_matrix {
    int n, log;
    vector<vector<int>>> pref;
    vector<int> mid;

    wavelet_matrix(vector<int> a): n(a.size()),
        log(_lg(*max_element(a.begin(), a.end()) | 1)), pref(log +
        1, vector<int>(n + 1)), mid(log + 1) {
        for (int b = log; b >= 0; --b) {
            for (int i = 0; i < n; ++i) pref[b][i + 1] = pref[b][i] +
                (((a[i] >> b) & 1) == 0);
            mid[b] = stable_partition(a.begin(), a.end(), [&](int x)
                {return ((x >> b) & 1) == 0;}) - a.begin();
        }
    }

    int kth(int i, int j, int k) {
        int x = 0;
        for (int b = log; b >= 0; --b) {
            int l = pref[b][i], r = pref[b][j + 1], z = r - l;
            if (k < z) i = l, j = r - 1;
            else x |= 1 << b, i += mid[b] - l, j += mid[b] - r, k -=
                z;
        }
        return x;
    }

    int less_or_equal(int i, int j, int x) {
        int c = 0;
        if (x >= (1 << (log + 1))) x = (1 << (log + 1)) - 1;
        for (int b = log; b >= 0; --b) {
            int l = pref[b][i], r = pref[b][j + 1], z = r - l;
            if (((x >> b) & 1) == 0) i = l, j = r - 1;
            else c += z, i += mid[b] - l, j += mid[b] - r;
        }
        if (i <= j) c += j - i + 1;
        return c;
    }
};

```

```

#define read(...) ([&](auto&... args) {(cin >> ... >>
    args);})(__VA_ARGS__)
#define print(...) ([&](auto&&... args) {(cout << ... <<
    args);})(__VA_ARGS__)

```

12 extra

12.1 random.cpp

```

mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());
uniform_int_distribution<int> dist(1, r);

```

12.2 permutation-int.cpp

```

int permutation_to_int(vector<int> &p) {
    int mask = 0;
    int i = 0, r = 0;
    for (int x : p) {
        r = r * ++i + __builtin_popcountll(mask & ~(1LL << x));
        mask |= 1LL << x;
    }
    return r;
}

```

12.3 pragmas.cpp

```

#pragma GCC optimize("O3,unroll-loops")
#pragma GCC target("avx2,bmi,bmi2,lzcnt,popcnt")

```

12.4 submasks.cpp

```

void submasks(int mask) {
    for (int submask = mask; submask > 0; submask = mask & (submask -
        1)) {
        process(submask);
    }
    process(0);
}

```

12.5 offset.cpp

```

template <typename T>
struct offset {
    int l;
    vector<T> a;

    offset(int l, int r, T x = T()): l(l), a(r - l, x) {}

    T &operator[](int i) {
        return a[i - l];
    }

    const T &operator[](int i) const {
        return a[i - l];
    }
};

```

12.6 macros.cpp